



US 20250272602A1

(19) **United States**

(12) **Patent Application Publication**
Kolli et al.

(10) **Pub. No.: US 2025/0272602 A1**

(43) **Pub. Date: Aug. 28, 2025**

(54) **ARTIFICIAL INTELLIGENCE TRAINING
USING ACCESIBILITY DATA**

(71) Applicant: **Oracle International Corporation**,
Redwood Shores, CA (US)

(72) Inventors: **Rajani Kolli**, San Jose, CA (US); **Dan
Foley**, Redmond, WA (US); **Pritesh
Kothari**, Fremont, CA (US)

(21) Appl. No.: **18/585,764**

(22) Filed: **Feb. 23, 2024**

Publication Classification

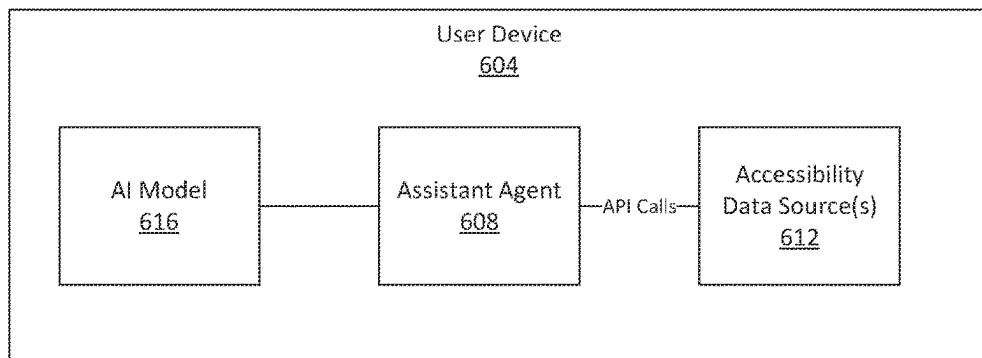
(51) **Int. Cl.**
G06N 20/00 (2019.01)

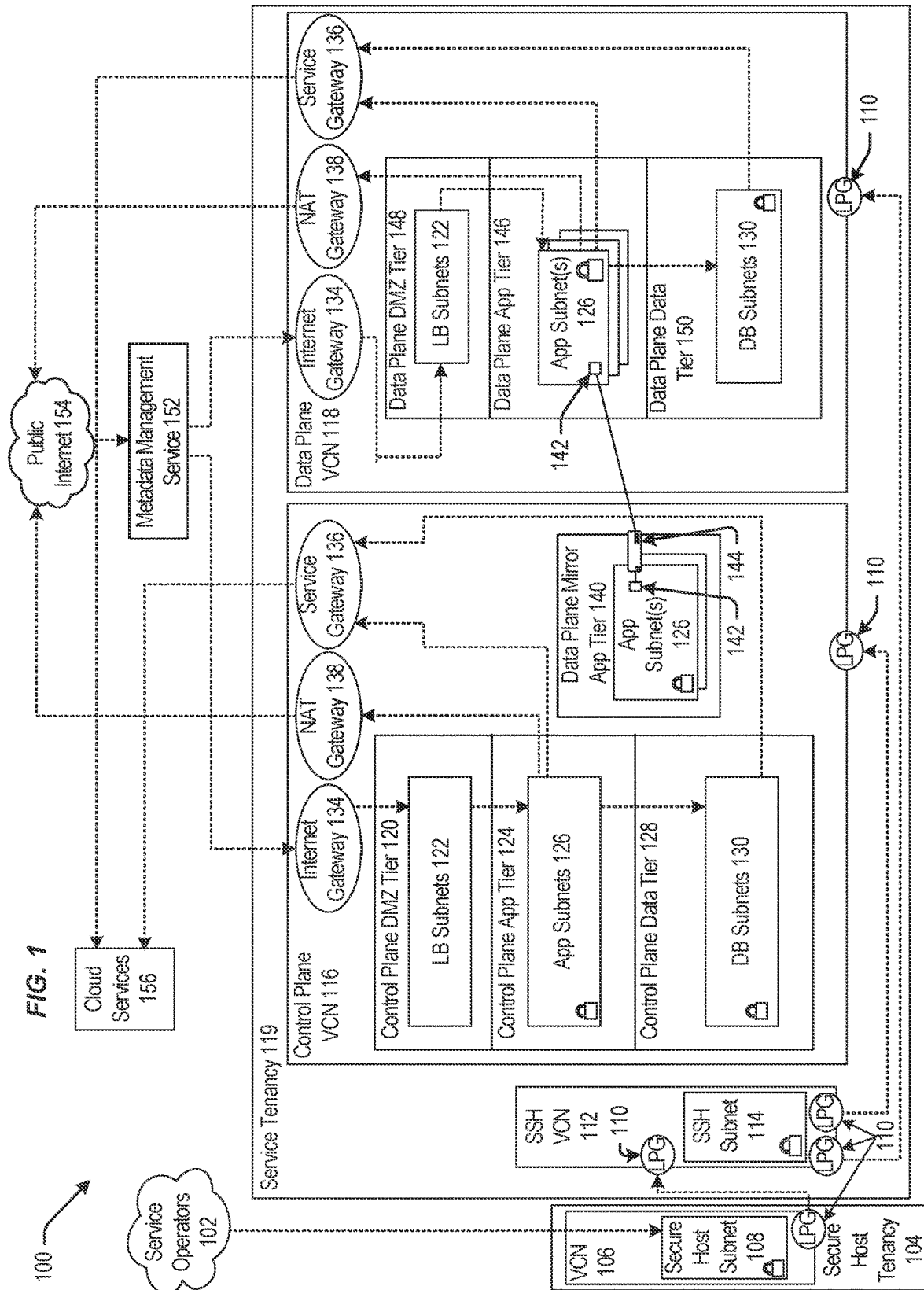
(52) **U.S. Cl.**
CPC **G06N 20/00** (2019.01)

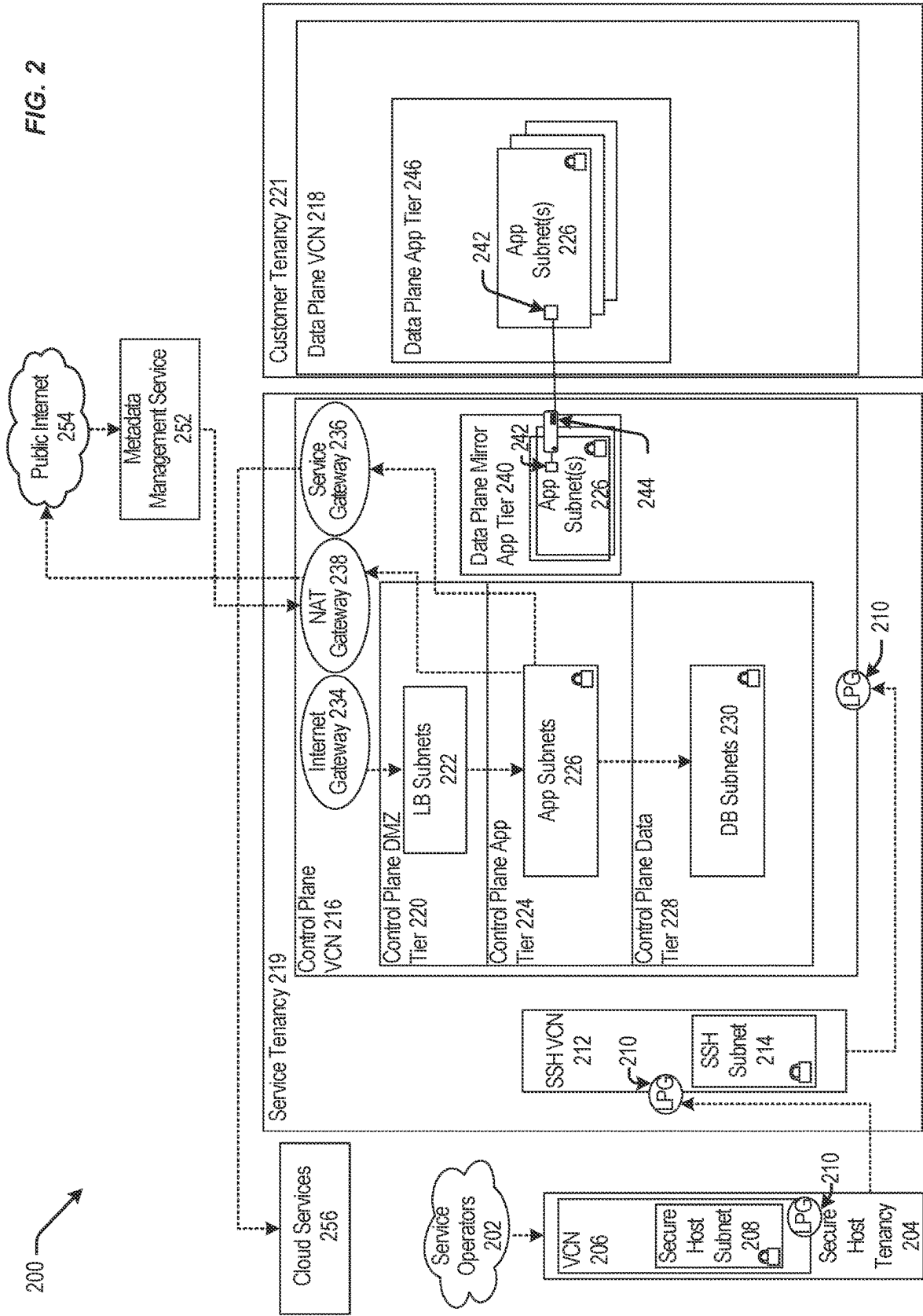
(57) **ABSTRACT**

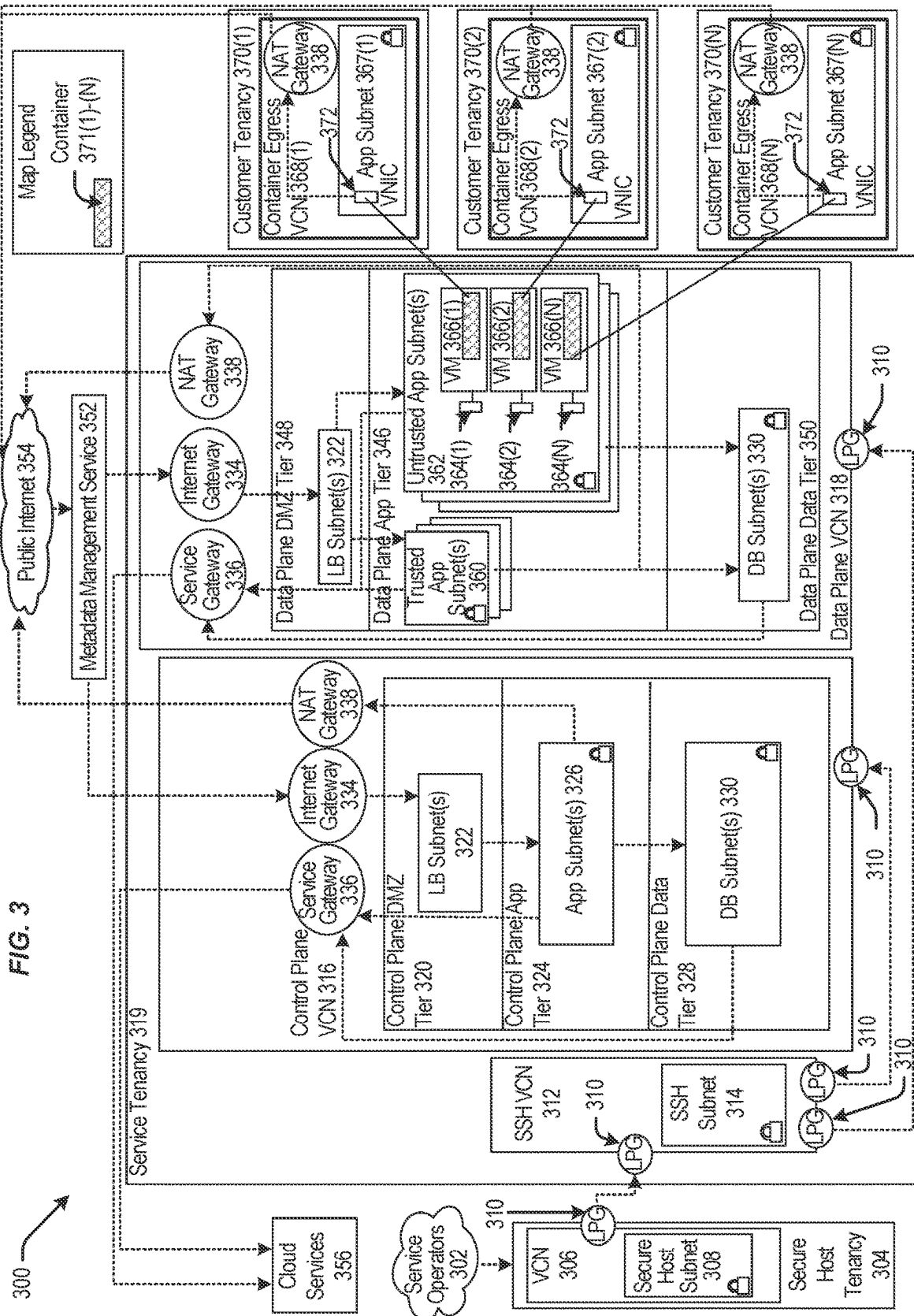
Examples provide an electronic device including at least one electronic processor configured to receive, via an accessibility application programming interface (“API”), first accessibility data associated with a first user interface (“UI”) displayed by a user device and including information identifying one or more UI elements and/or UI events in the first UI; identify, based on the first accessibility data, a first action performed by a user of the user device and defining a first set of operations; train, based on the first accessibility data and the first set of operations, an artificial intelligence (“AI”) model; receive, via the accessibility API, second accessibility data associated with a second UI displayed by the user device; determine, based on the second accessibility data and the AI model, a recommended action defining a second set of operations; and output a command to perform the recommended action on the user device.

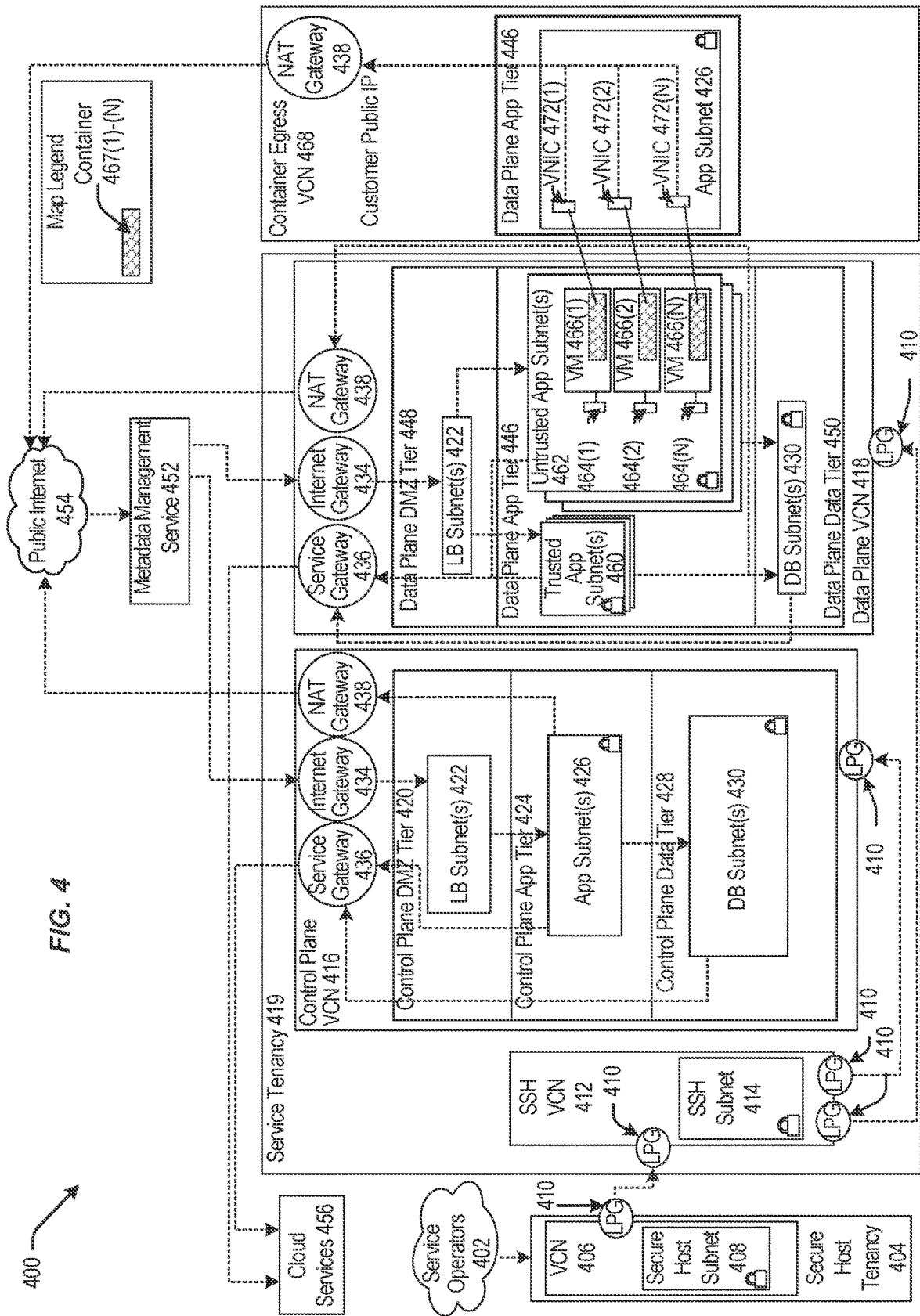
600











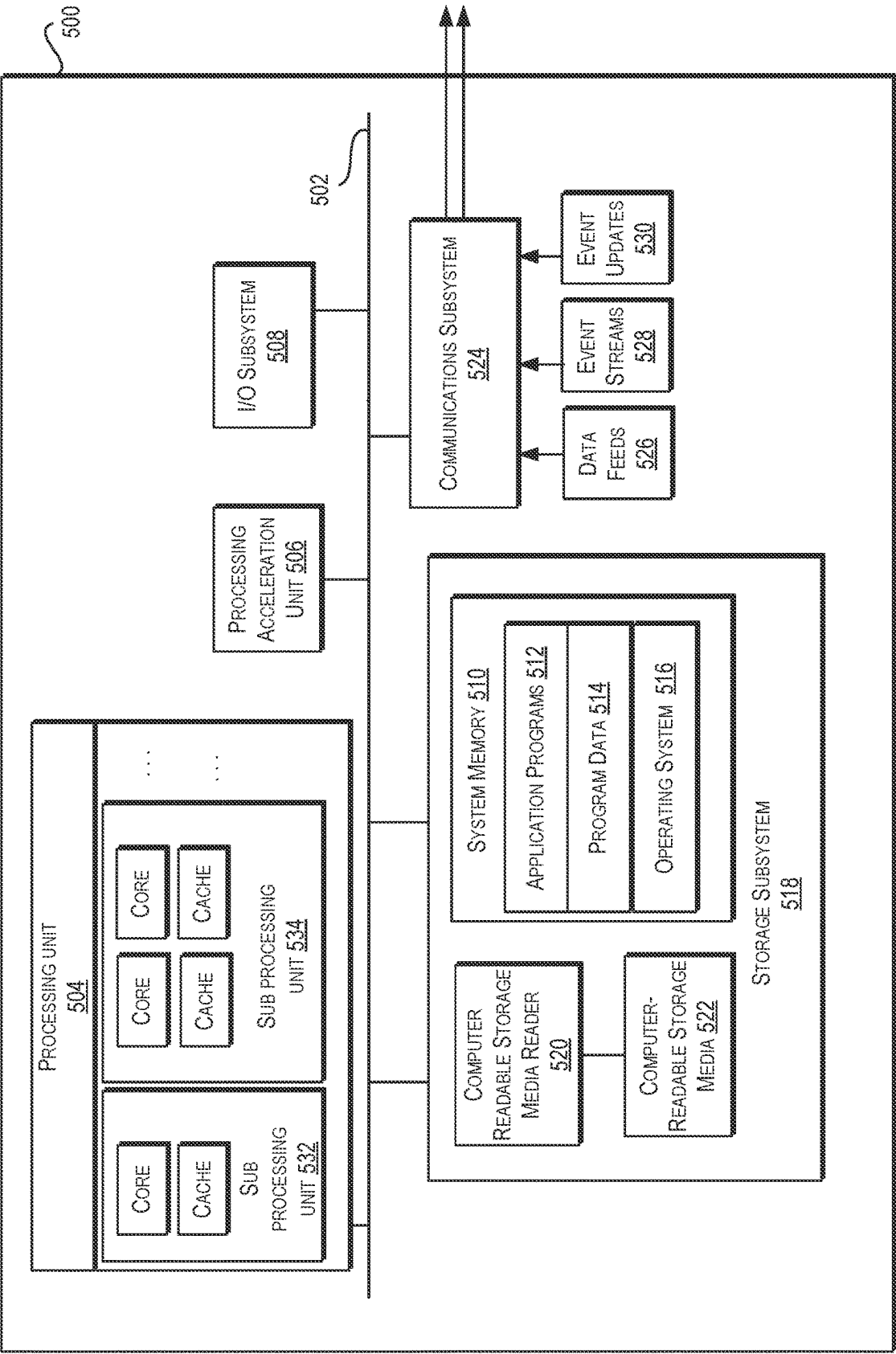


FIG. 5

600

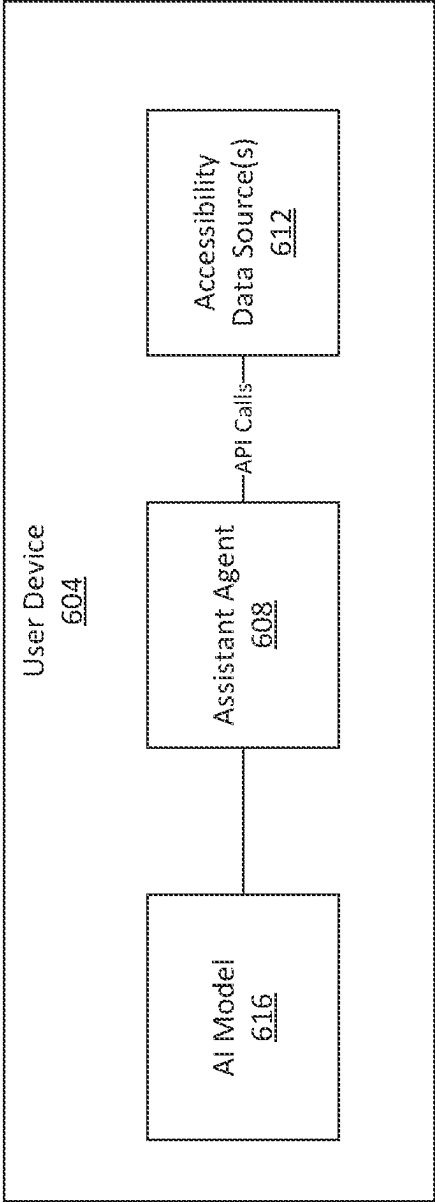


FIG. 6

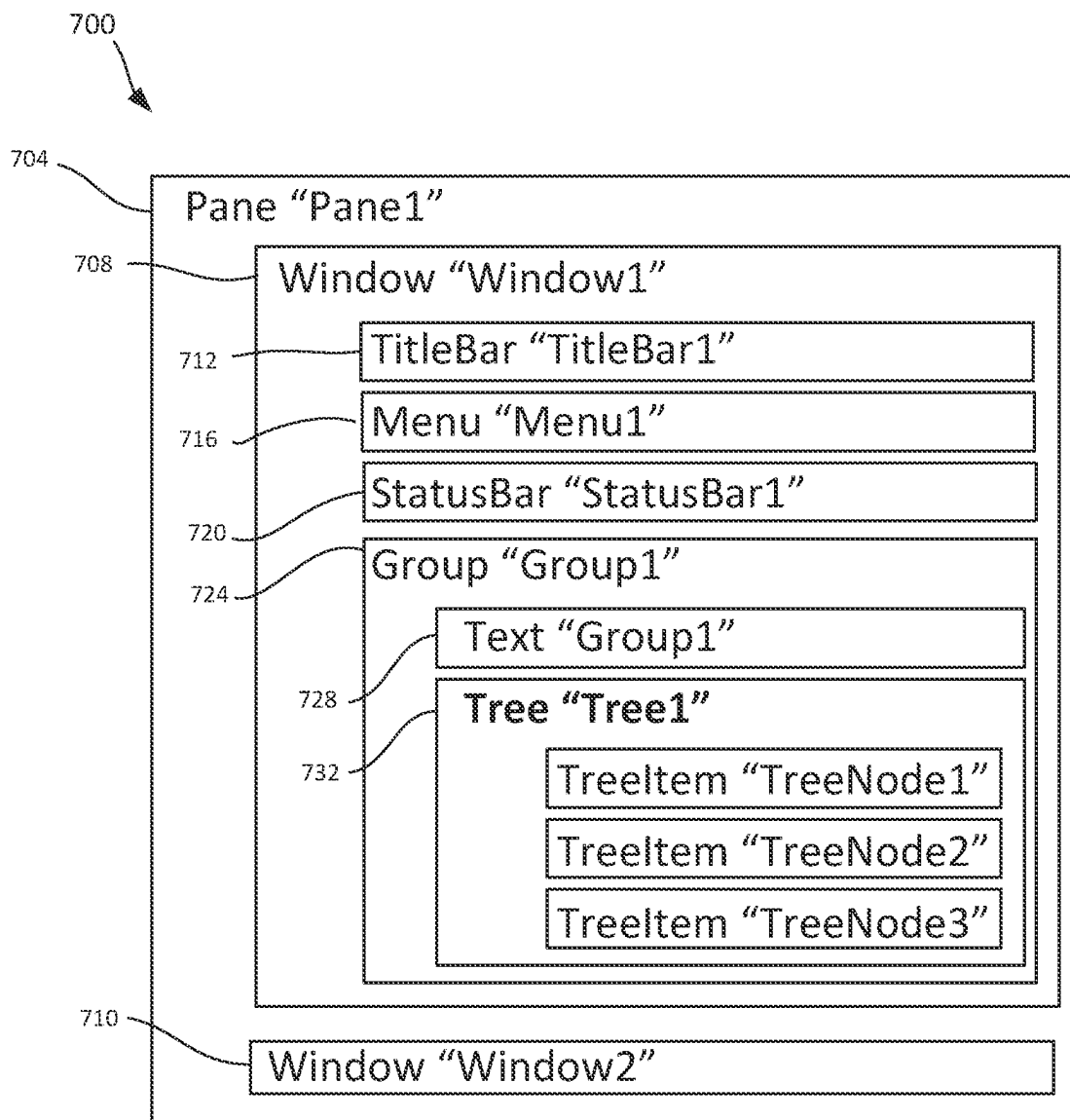


FIG. 7A

740



Name: Tree1
ClassName: TreeView
ControlType: Tree
LocalizedControlType: Control View
FrameworkType: Wpf
FrameworkID: WPF
ProcessID: 24132
BoundingRectnagle: X=30, Y=84, Width= 480, Height = 502
AnnotationPattern: no
DockPattern: no
DragPattern: no
DropTargetPattern: no
ExpandCollapsePattern: no
GridItemPattern: no
InvokePattern: no
ItemContainerPattern: yes
LegacyIAccessiblePattern: no
MultipleViewPattern: no
ObjectModelPattern: no

FIG. 7B

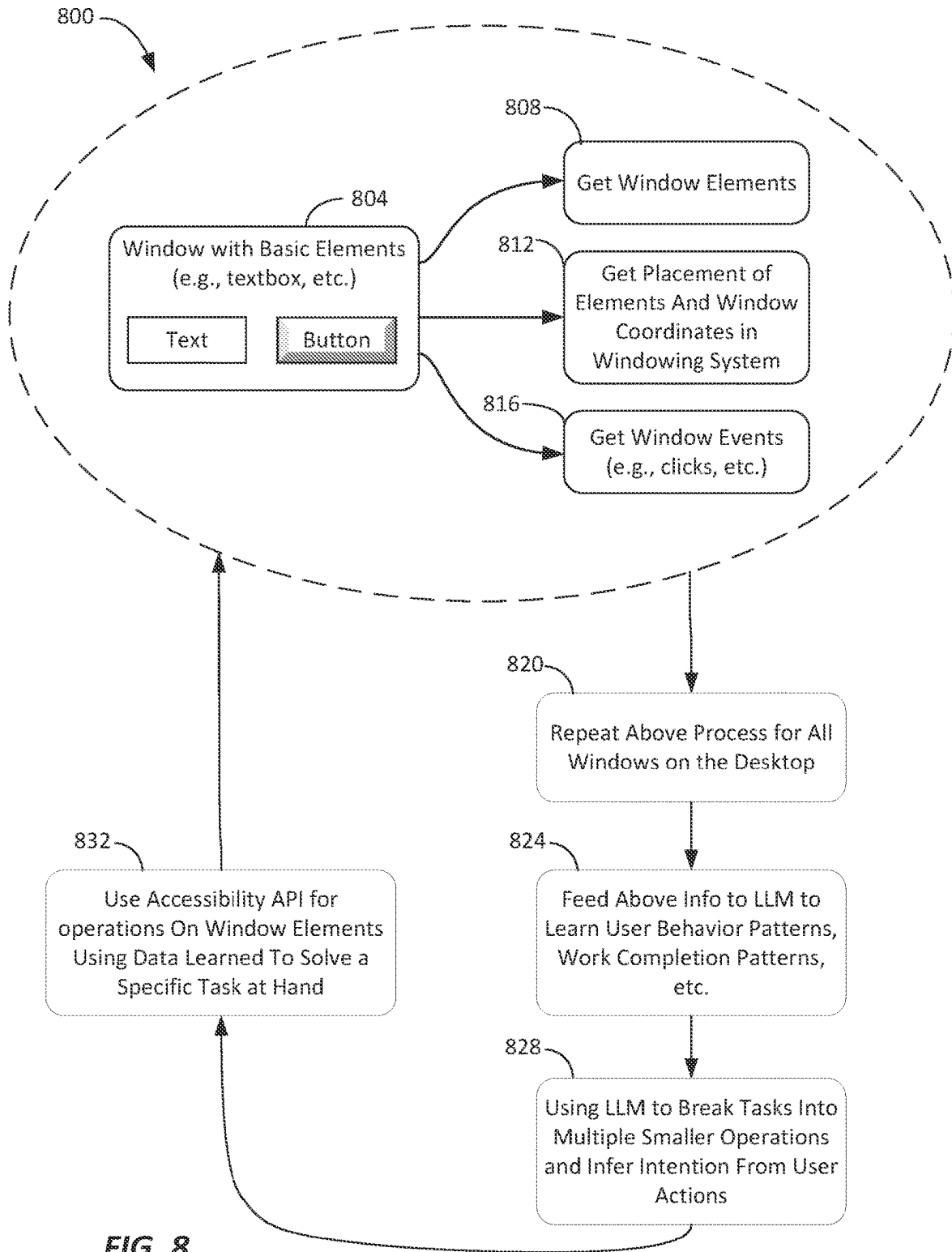
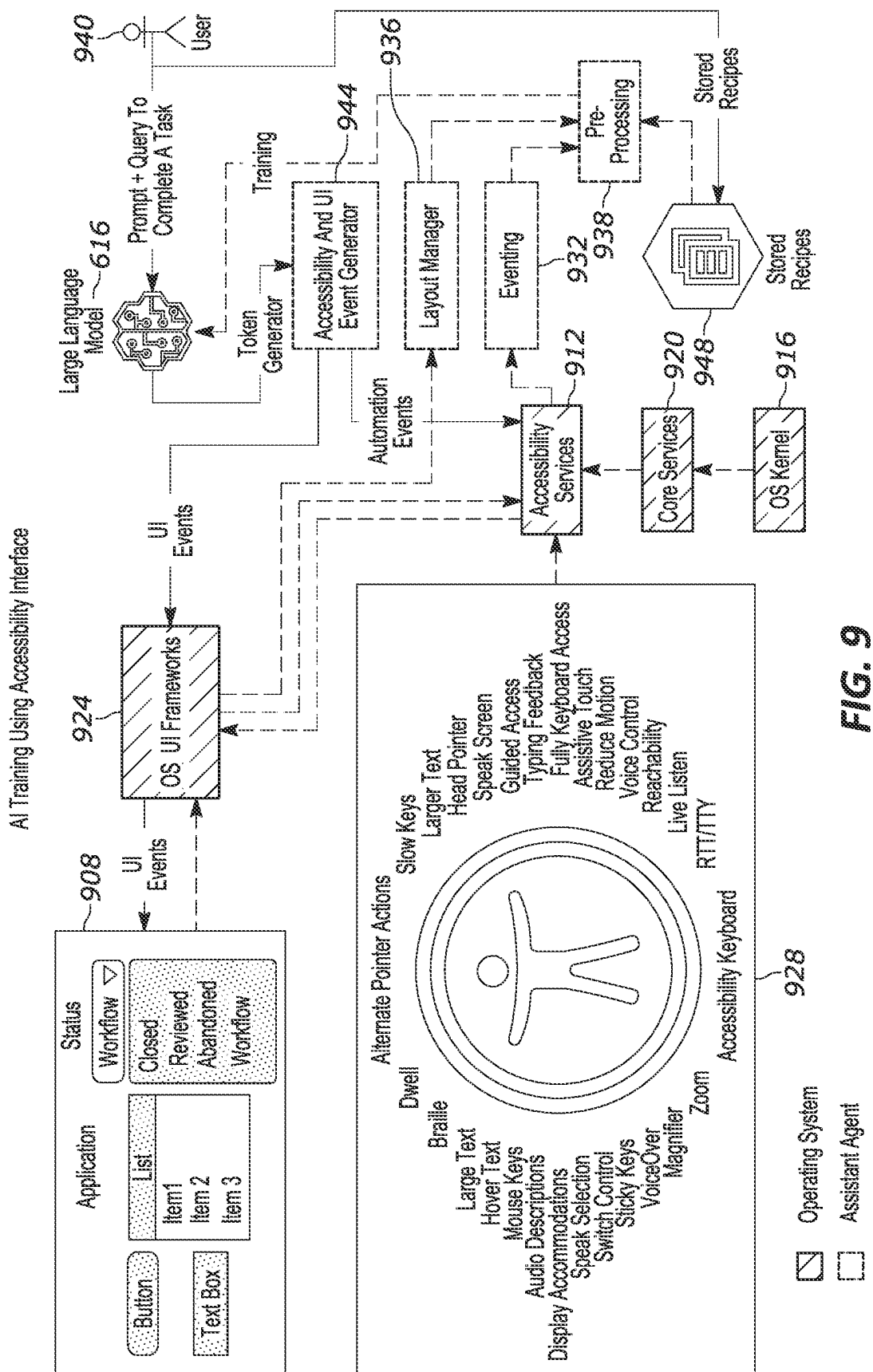


FIG. 8



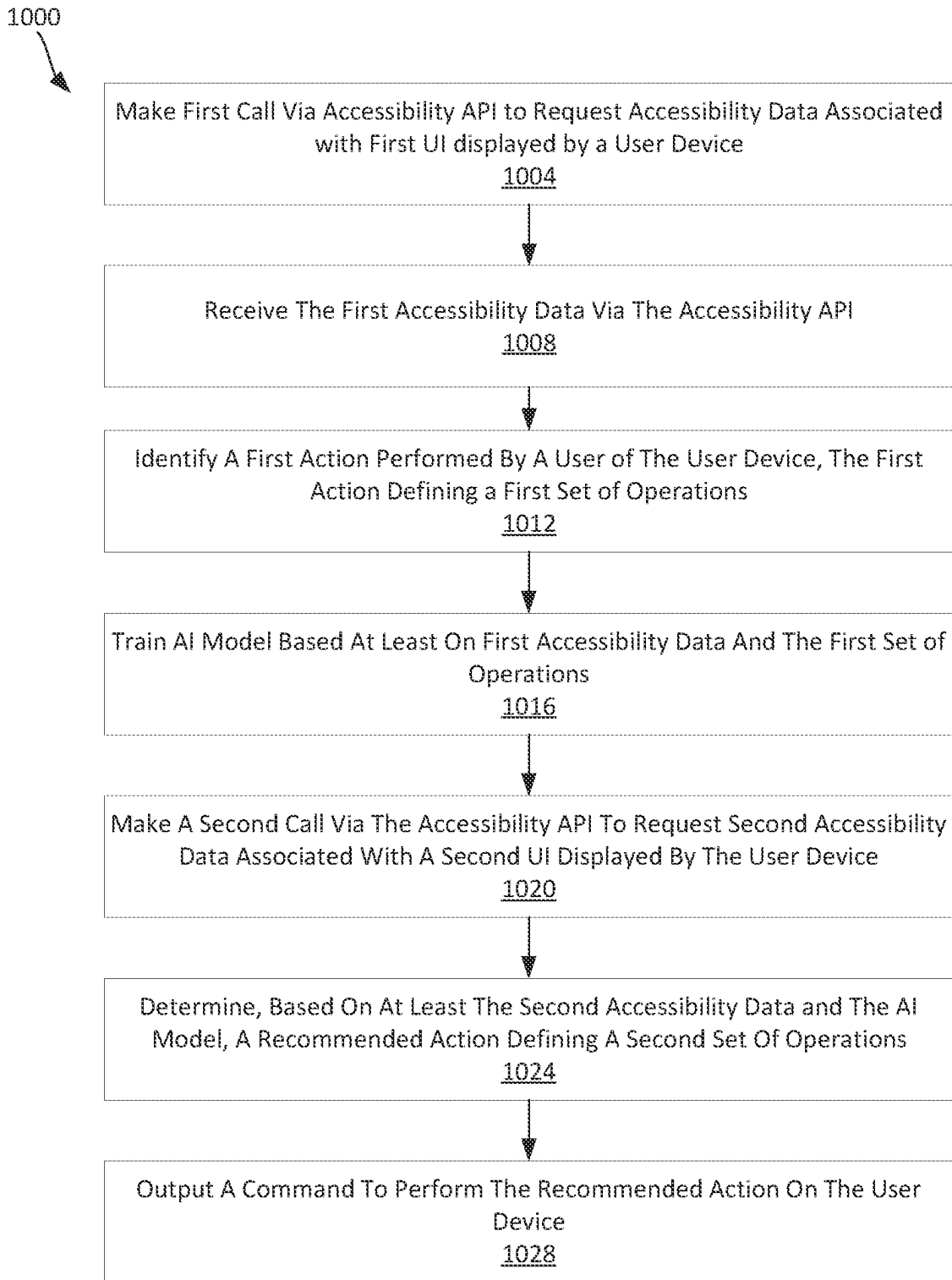


FIG. 10

ARTIFICIAL INTELLIGENCE TRAINING USING ACCESSIBILITY DATA

FIELD

[0001] Embodiments described herein generally relate to training artificial intelligence models.

SUMMARY

[0002] Conventional assistant artificial intelligence (“AI”) models are trained in a manner similar to AI chat bots using backend data organized into troubleshooting trees and such models cannot view what the user sees and does on user devices. As a result, the scope, accuracy, and efficiency of tasks that can be performed by conventional assistant AI models is limited. For example, such AI models may not be trained with data representing the context surrounding a task performed by a user. As one example, a particular task performed by a user may be defined by a particular set of operations in a first context but may be defined by a different set of operations when the task is performed in a second, different context, such as another date or time, another work session, or when applications currently executed on the user device vary. Thus, recording actions merely allows an assistant AI model or other type of task assistant to repeat a defined set of operations and does not infer the intention (i.e., the intended result) of those operations. Further, recorded actions can generally only be played back when the same window elements are active (e.g., the same applications are executing on the user device and have the same configuration of user interface elements) as when the action was recorded. Additionally, action recording is often only available on a per-application basis. In other words, a user may not be able to record actions that interact with multiple different applications.

[0003] Accordingly, the operations required to perform an intended task often change based on context. Thus, there is a need for improved training of assistant AI models using data from, for example, an accessibility interface to learn a wide variety of specific tasks based on the intended result of those tasks, user behavior patterns, work completion patterns, and the like. This solution allows the AI model to adapt performance of tasks according to changes in context surrounding the functions.

[0004] One example provides an electronic device including at least one electronic processor configured to: make a first call via an accessibility application programming interface (“API”) to request first accessibility data associated with a first user interface (“UI”) displayed by a user device; receive the first accessibility data via the accessibility API, the first accessibility data comprising at least one of (a) information identifying one or more UI elements in the first UI or (b) information identifying one or more UI events in the first UI; identify, based on at least the first accessibility data, a first action performed by a user of the user device, the first action defining a first set of operations; train, based on at least the first accessibility data and the first set of operations, an artificial intelligence (“AI”) model; make a second call via the accessibility API to request second accessibility data associated with a second UI displayed by the user device; determine, based on at least the second accessibility data and the AI model, a recommended action,

the recommend action defining a second set of operations; and output a command to perform the recommended action on the user device.

[0005] In some aspects, the first set of operations differs from the second set of operations.

[0006] In some aspects, the at least one electronic processor is configured to output the command to perform the recommended action by making a third call via the accessibility API to the user device.

[0007] In some aspects, the at least one electronic processor is configured to output the command to at least one application executed on the user device.

[0008] In some aspects, the at least one electronic processor is configured to output the command by making a call to an API associated with the at least one application executed on the user device.

[0009] In some aspects, the information identifying the one or more UI elements in the first UI includes at least one selected from the group consisting of: (a) a respective element type of the one or more UI elements in the first UI, (b) a respective identifier of the one or more UI elements in the first UI, or (c) a respective state or condition of the one or more UI elements in the first UI.

[0010] In some aspects, the state or condition of the one or more UI elements in the first UI includes at least one selected from the group consisting of pixel coordinates of the one or more UI elements, dimensions of the one or more UI elements, and an active or passive state of the one or more UI elements.

[0011] In some aspects, the information identifying the one or more UI elements in the first UI is organized as a hierarchical tree.

[0012] In some aspects, the one or more UI elements in the first UI comprises a first UI element that includes a second UI element; the hierarchical tree comprises a first hierarchical level that is above a second hierarchical level; the first hierarchical level includes a first node representing the first UI element; and the second hierarchical level includes a second node representing the second UI element.

[0013] In some aspects, the information identifying the one or more UI events in the first UI includes one or more notifications of changes in states or conditions of the UI elements in the first UI.

[0014] In some aspects, the changes in states or conditions of the UI elements in the first UI are caused by user interaction with the UI elements.

[0015] In some aspects, the first call made via the accessibility API is made to a platform rendering the first UI.

[0016] In some aspects, the platform includes one of an operating system executing on the first device or a browser application executing on the first device.

[0017] In some aspects, the first accessibility data is generated based on metadata associated with the one or more UI elements in the first UI, the metadata being exposed via the accessibility API.

[0018] In some aspects, the metadata is specified in one of (a) application code of application for which the first UI is being rendered or (b) content code of web content for which the first UI is being rendered.

[0019] In some aspects, the metadata specifies hierarchical relationships between the one or more UI elements in the first UI.

[0020] In some aspects, the metadata of the first UI elements includes at least one selected from a group con-

sisting of a type of each respective first UI element, an active or passive status of each respective first UI element, pixel coordinates of each respective first UI element, and a displayed label of each respective first UI element.

[0021] In some aspects, the first accessibility data is generated by a platform executing on the first device during rendering of the first UI.

[0022] Another example provides a method for training an artificial intelligence model. The method includes: making a first call via an accessibility application programming interface (“API”) to request first accessibility data associated with a first user interface (“UI”) displayed by a user device; receiving the first accessibility data via the accessibility API, the first accessibility data comprising at least one of (a) information identifying one or more UI elements in the first UI or (b) information identifying one or more UI events in the first UI; identifying, based on at least the first accessibility data, a first action performed by a user of the user device, the first action defining a first set of operations; training, based on at least the first accessibility data and the first set of operations, an artificial intelligence (“AI”) model; making a second call via the accessibility API to request second accessibility data associated with a second UI displayed by the user device; determining, based on at least the second accessibility data and the AI model, a recommended action, the recommended action defining a second set of operations; and outputting a command to perform the recommended action on the user device.

[0023] In some aspects, the first set of operations differs from the second set of operations.

[0024] Other aspects will become apparent by consideration of the detailed description and accompanying drawings.

BRIEF DESCRIPTION OF THE DRAWINGS

[0025] FIG. 1 is a block diagram illustrating one pattern for implementing a cloud infrastructure as a service system, according to at least one embodiment.

[0026] FIG. 2 is a block diagram illustrating another pattern for implementing a cloud infrastructure as a service system, according to at least one embodiment.

[0027] FIG. 3 is a block diagram illustrating another pattern for implementing a cloud infrastructure as a service system, according to at least one embodiment.

[0028] FIG. 4 is a block diagram illustrating another pattern for implementing a cloud infrastructure as a service system, according to at least one embodiment.

[0029] FIG. 5 is a block diagram illustrating an example computer system, according to at least one embodiment.

[0030] FIG. 6 is a block diagram illustrating a system architecture for training an artificial intelligence model to perform tasks, according to at least one embodiment.

[0031] FIG. 7A illustrates an example accessibility data tree, according to at least one embodiment.

[0032] FIG. 7B illustrates an example details portion of a selected UI element of the accessibility data tree of FIG. 7A, according to at least one embodiment.

[0033] FIG. 8 is a flowchart illustrating an example method for training an artificial intelligence model to perform tasks, according to at least one embodiment.

[0034] FIG. 9 is a diagram illustrating a workflow for training an artificial intelligence model to perform tasks, according to at least one embodiment.

[0035] FIG. 10 is a flowchart illustrating a method for training an artificial intelligence model to perform tasks, according to at least one example.

DETAILED DESCRIPTION

[0036] In the following description, various embodiments will be described. For purposes of explanation, specific configurations and details are set forth to provide a thorough understanding of the embodiments. However, it will also be apparent to one skilled in the art that the embodiments may be practiced without the specific details. Furthermore, well-known features may be omitted or simplified in order not to obscure the embodiment being described.

Cloud-Based Computing Platforms

[0037] Embodiments described herein may be performed, wholly or partly, within a cloud-based computing platform. Cloud-based computing platforms provide scalable and flexible computing resources for users. Infrastructure as a service (IaaS) is one particular type of cloud computing. IaaS can be configured to provide virtualized computing resources over a public network (e.g., the Internet). In an IaaS model, a cloud computing provider can host the infrastructure components (e.g., servers, storage devices, network nodes (e.g., hardware), deployment software, platform virtualization (e.g., a hypervisor layer), or the like). In some cases, an IaaS provider may also supply a variety of services to accompany those infrastructure components (example services include billing software, monitoring software, logging software, load balancing software, clustering software, etc.). Thus, as these services may be policy-driven, IaaS users may be able to implement policies to drive load balancing to maintain application availability and performance.

[0038] In some instances, IaaS customers may access resources and services through a wide area network (WAN), such as the Internet, and can use the cloud provider’s services to install the remaining elements of an application stack. For example, the user can log in to the IaaS platform to create virtual machines (VMs), install operating systems (OSs) on each VM, deploy middleware such as databases, create storage buckets for workloads and backups, and even install enterprise software into that VM. Customers can then use the provider’s services to perform various functions, including balancing network traffic, troubleshooting application issues, monitoring performance, managing disaster recovery, etc.

[0039] In most cases, a cloud computing model will require the participation of a cloud provider. The cloud provider may, but need not be, a third-party service that specializes in providing (e.g., offering, renting, selling) IaaS. An entity might also opt to deploy a private cloud, becoming its own provider of infrastructure services.

[0040] In some examples, IaaS deployment is the process of putting a new application, or a new version of an application, onto a prepared application server or the like. It may also include the process of preparing the server (e.g., installing libraries, daemons, etc.). This is often managed by the cloud provider, below the hypervisor layer (e.g., the servers, storage, network hardware, and virtualization). Thus, the customer may be responsible for handling (OS),

middleware, and/or application deployment (e.g., on self-service virtual machines (e.g., that can be spun up on demand) or the like.

[0041] In some examples, IaaS provisioning may refer to acquiring computers or virtual hosts for use, and even installing needed libraries or services on them. In most cases, deployment does not include provisioning, and the provisioning may need to be performed first.

[0042] In some cases, there are two different challenges for IaaS provisioning. First, there is the initial challenge of provisioning the initial set of infrastructure before anything is running. Second, there is the challenge of evolving the existing infrastructure (e.g., adding new services, changing services, removing services, etc.) once everything has been provisioned. In some cases, these two challenges may be addressed by enabling the configuration of the infrastructure to be defined declaratively. In other words, the infrastructure (e.g., what components are needed and how they interact) can be defined by one or more configuration files. Thus, the overall topology of the infrastructure (e.g., what resources depend on which, and how they each work together) can be described declaratively. In some instances, once the topology is defined, a workflow can be generated that creates and/or manages the different components described in the configuration files.

[0043] In some examples, an infrastructure may have many interconnected elements. For example, there may be one or more virtual private clouds (VPCs) (e.g., a potentially on-demand pool of configurable and/or shared computing resources), also known as a core network. In some examples, there may also be one or more inbound/outbound traffic group rules provisioned to define how the inbound and/or outbound traffic of the network will be set up and one or more virtual machines (VMs). Other infrastructure elements may also be provisioned, such as a load balancer, a database, or the like. As more and more infrastructure elements are desired and/or added, the infrastructure may incrementally evolve.

[0044] In some instances, continuous deployment techniques may be employed to enable deployment of infrastructure code across various virtual computing environments. Additionally, the described techniques can enable infrastructure management within these environments. In some examples, service teams can write code that is desired to be deployed to one or more, but often many, different production environments (e.g., across various different geographic locations, sometimes spanning the entire world). However, in some examples, the infrastructure on which the code will be deployed must first be set up. In some instances, the provisioning can be done manually, a provisioning tool may be utilized to provision the resources, and/or deployment tools may be utilized to deploy the code once the infrastructure is provisioned.

[0045] FIG. 1 is a block diagram 100 illustrating an example pattern of an IaaS architecture, according to at least one embodiment. Service operators 102 can be communicatively coupled to a secure host tenancy 104 that can include a virtual cloud network (VCN) 106 and a secure host subnet 108. In some examples, the service operators 102 may use one or more client computing devices, which may be portable handheld devices (e.g., an iPhone®, cellular telephone, an iPad®, computing tablet, a personal digital assistant (PDA)) or wearable devices (e.g., a Google Glass® head mounted display), running software and/or a variety of

mobile operating systems such as iOS, Windows Phone, Android, BlackBerry 8, Palm OS, and the like, and being Internet, e-mail, short message service (SMS), BlackBerry®, or other communication protocol enabled. Alternatively, the client computing devices can be general purpose personal computers including, by way of example, personal computers and/or laptop computers running various versions of Microsoft Windows®, Apple Macintosh®, and/or Linux operating systems. The client computing devices can be workstation computers running any of a variety of commercially-available UNIX® or UNIX-like operating systems, including without limitation the variety of GNU/Linux operating systems, such as for example, Google Chrome OS. Alternatively, or in addition, client computing devices may be any other electronic device, such as a thin-client computer, an Internet-enabled gaming system (e.g., a Microsoft Xbox gaming console with or without a Kinect® gesture input device), and/or a personal messaging device, capable of communicating over a network that can access the VCN 106 and/or the Internet.

[0046] The VCN 106 can include a local peering gateway (LPG) 110 that can be communicatively coupled to a secure shell (SSH) VCN 112 via an LPG 110 contained in the SSH VCN 112. The SSH VCN 112 can include an SSH subnet 114, and the SSH VCN 112 can be communicatively coupled to a control plane VCN 116 via the LPG 110 contained in the control plane VCN 116. Also, the SSH VCN 112 can be communicatively coupled to a data plane VCN 118 via an LPG 110. The control plane VCN 116 and the data plane VCN 118 can be contained in a service tenancy 119 that can be owned and/or operated by the IaaS provider.

[0047] The control plane VCN 116 can include a control plane demilitarized zone (DMZ) tier 120 that acts as a perimeter network (e.g., portions of a corporate network between the corporate intranet and external networks). The DMZ-based servers may have restricted responsibilities and help keep breaches contained. Additionally, the DMZ tier 120 can include one or more load balancer (LB) subnet(s) 122, a control plane app tier 124 that can include app subnet(s) 126, a control plane data tier 128 that can include database (DB) subnet(s) 130 (e.g., frontend DB subnet(s) and/or backend DB subnet(s)). The LB subnet(s) 122 contained in the control plane DMZ tier 120 can be communicatively coupled to the app subnet(s) 126 contained in the control plane app tier 124 and an Internet gateway 134 that can be contained in the control plane VCN 116, and the app subnet(s) 126 can be communicatively coupled to the DB subnet(s) 130 contained in the control plane data tier 128 and a service gateway 136 and a network address translation (NAT) gateway 138. The control plane VCN 116 can include the service gateway 136 and the NAT gateway 138.

[0048] The control plane VCN 116 can include a data plane mirror app tier 140 that can include app subnet(s) 126. The app subnet(s) 126 contained in the data plane mirror app tier 140 can include a virtual network interface controller (VNIC) 142 that can execute a compute instance 144. The compute instance 144 can communicatively couple the app subnet(s) 126 of the data plane mirror app tier 140 to app subnet(s) 126 that can be contained in a data plane app tier 146.

[0049] The data plane VCN 118 can include the data plane app tier 146, a data plane DMZ tier 148, and a data plane data tier 150. The data plane DMZ tier 148 can include LB subnet(s) 122 that can be communicatively coupled to the

app subnet(s) 126 of the data plane app tier 146 and the Internet gateway 134 of the data plane VCN 118. The app subnet(s) 126 can be communicatively coupled to the service gateway 136 of the data plane VCN 118 and the NAT gateway 138 of the data plane VCN 118. The data plane data tier 150 can also include the DB subnet(s) 130 that can be communicatively coupled to the app subnet(s) 126 of the data plane app tier 146.

[0050] The Internet gateway 134 of the control plane VCN 116 and of the data plane VCN 118 can be communicatively coupled to a metadata management service 152 that can be communicatively coupled to public Internet 154. Public Internet 154 can be communicatively coupled to the NAT gateway 138 of the control plane VCN 116 and of the data plane VCN 118. The service gateway 136 of the control plane VCN 116 and of the data plane VCN 118 can be communicatively coupled to cloud services 156.

[0051] In some examples, the service gateway 136 of the control plane VCN 116 or of the data plane VCN 118 can make application programming interface (API) calls to cloud services 156 without going through public Internet 154. The API calls to cloud services 156 from the service gateway 136 can be one-way: the service gateway 136 can make API calls to cloud services 156, and cloud services 156 can send requested data to the service gateway 136. But, cloud services 156 may not initiate API calls to the service gateway 136.

[0052] In some examples, the secure host tenancy 104 can be directly connected to the service tenancy 119, which may be otherwise isolated. The secure host subnet 108 can communicate with the SSH subnet 114 through an LPG 110 that may enable two-way communication over an otherwise isolated system. Connecting the secure host subnet 108 to the SSH subnet 114 may give the secure host subnet 108 access to other entities within the service tenancy 119.

[0053] The control plane VCN 116 may allow users of the service tenancy 119 to set up or otherwise provision desired resources. Desired resources provisioned in the control plane VCN 116 may be deployed or otherwise used in the data plane VCN 118. In some examples, the control plane VCN 116 can be isolated from the data plane VCN 118, and the data plane mirror app tier 140 of the control plane VCN 116 can communicate with the data plane app tier 146 of the data plane VCN 118 via VNICs 142 that can be contained in the data plane mirror app tier 140 and the data plane app tier 146.

[0054] In some examples, users of the system can make requests, for example create, read, update, or delete (CRUD) operations, through public Internet 154 that can communicate the requests to the metadata management service 152. The metadata management service 152 can communicate the request to the control plane VCN 116 through the Internet gateway 134. The request can be received by the LB subnet(s) 122 contained in the control plane DMZ tier 120. The LB subnet(s) 122 may determine that the request is valid, and in response to this determination, the LB subnet(s) 122 can transmit the request to app subnet(s) 126 contained in the control plane app tier 124. If the request is validated and requires a call to public Internet 154, the call to public Internet 154 may be transmitted to the NAT gateway 138 that can make the call to public Internet 154. Metadata that may be desired to be stored by the request can be stored in the DB subnet(s) 130.

[0055] In some examples, the data plane mirror app tier 140 can facilitate direct communication between the control plane VCN 116 and the data plane VCN 118. For example, changes, updates, or other suitable modifications to configuration may be desired to be applied to the resources contained in the data plane VCN 118. Via a VNIC 142, the control plane VCN 116 can directly communicate with, and can thereby execute the changes, updates, or other suitable modifications to configuration to, resources contained in the data plane VCN 118.

[0056] In some embodiments, the control plane VCN 116 and the data plane VCN 118 can be contained in the service tenancy 119. In this case, the user, or the customer, of the system may not own or operate either the control plane VCN 116 or the data plane VCN 118. Instead, the IaaS provider may own or operate the control plane VCN 116 and the data plane VCN 118, both of which may be contained in the service tenancy 119. This embodiment can enable isolation of networks that may prevent users or customers from interacting with other users', or other customers', resources. Also, this embodiment may allow users or customers of the system to store databases privately without needing to rely on public Internet 154, which may not have a desired level of threat prevention, for storage.

[0057] In other embodiments, the LB subnet(s) 122 contained in the control plane VCN 116 can be configured to receive a signal from the service gateway 136. In this embodiment, the control plane VCN 116 and the data plane VCN 118 may be configured to be called by a customer of the IaaS provider without calling public Internet 154. Customers of the IaaS provider may desire this embodiment since database(s) that the customers use may be controlled by the IaaS provider and may be stored on the service tenancy 119, which may be isolated from public Internet 154.

[0058] FIG. 2 is a block diagram 200 illustrating another example pattern of an IaaS architecture, according to at least one embodiment. Service operators 202 (e.g., service operators 102 of FIG. 1) can be communicatively coupled to a secure host tenancy 204 (e.g., the secure host tenancy 104 of FIG. 1) that can include a virtual cloud network (VCN) 206 (e.g., the VCN 106 of FIG. 1) and a secure host subnet 208 (e.g., the secure host subnet 108 of FIG. 1). The VCN 206 can include a local peering gateway (LPG) 210 (e.g., the LPG 110 of FIG. 1) that can be communicatively coupled to a secure shell (SSH) VCN 212 (e.g., the SSH VCN 112 of FIG. 1) via an LPG 110 contained in the SSH VCN 212. The SSH VCN 212 can include an SSH subnet 214 (e.g., the SSH subnet 114 of FIG. 1), and the SSH VCN 212 can be communicatively coupled to a control plane VCN 216 (e.g., the control plane VCN 116 of FIG. 1) via an LPG 210 contained in the control plane VCN 216. The control plane VCN 216 can be contained in a service tenancy 219 (e.g., the service tenancy 119 of FIG. 1), and the data plane VCN 218 (e.g., the data plane VCN 118 of FIG. 1) can be contained in a customer tenancy 221 that may be owned or operated by users, or customers, of the system.

[0059] The control plane VCN 216 can include a control plane DMZ tier 220 (e.g., the control plane DMZ tier 120 of FIG. 1) that can include LB subnet(s) 222 (e.g., LB subnet(s) 122 of FIG. 1), a control plane app tier 224 (e.g., the control plane app tier 124 of FIG. 1) that can include app subnet(s) 226 (e.g., app subnet(s) 126 of FIG. 1), a control plane data tier 228 (e.g., the control plane data tier 128 of FIG. 1) that

can include database (DB) subnet(s) **230** (e.g., similar to DB subnet(s) **130** of FIG. 1). The LB subnet(s) **222** contained in the control plane DMZ tier **220** can be communicatively coupled to the app subnet(s) **226** contained in the control plane app tier **224** and an Internet gateway **234** (e.g., the Internet gateway **134** of FIG. 1) that can be contained in the control plane VCN **216**, and the app subnet(s) **226** can be communicatively coupled to the DB subnet(s) **230** contained in the control plane data tier **228** and a service gateway **236** (e.g., the service gateway **136** of FIG. 1) and a network address translation (NAT) gateway **238** (e.g., the NAT gateway **138** of FIG. 1). The control plane VCN **216** can include the service gateway **236** and the NAT gateway **238**.

[0060] The control plane VCN **216** can include a data plane mirror app tier **240** (e.g., the data plane mirror app tier **140** of FIG. 1) that can include app subnet(s) **226**. The app subnet(s) **226** contained in the data plane mirror app tier **240** can include a virtual network interface controller (VNIC) **242** (e.g., the VNIC of **142**) that can execute a compute instance **244** (e.g., similar to the compute instance **144** of FIG. 1). The compute instance **244** can facilitate communication between the app subnet(s) **226** of the data plane mirror app tier **240** and the app subnet(s) **226** that can be contained in a data plane app tier **246** (e.g., the data plane app tier **146** of FIG. 1) via the VNIC **242** contained in the data plane mirror app tier **240** and the VNIC **242** contained in the data plane app tier **246**.

[0061] The Internet gateway **234** contained in the control plane VCN **216** can be communicatively coupled to a metadata management service **252** (e.g., the metadata management service **152** of FIG. 1) that can be communicatively coupled to public Internet **254** (e.g., public Internet **154** of FIG. 1). Public Internet **254** can be communicatively coupled to the NAT gateway **238** contained in the control plane VCN **216**. The service gateway **236** contained in the control plane VCN **216** can be communicatively coupled to cloud services **256** (e.g., cloud services **156** of FIG. 1).

[0062] In some examples, the data plane VCN **218** can be contained in the customer tenancy **221**. In this case, the IaaS provider may provide the control plane VCN **216** for each customer, and the IaaS provider may, for each customer, set up a unique compute instance **244** that is contained in the service tenancy **219**. Each compute instance **244** may allow communication between the control plane VCN **216**, contained in the service tenancy **219**, and the data plane VCN **218** that is contained in the customer tenancy **221**. The compute instance **244** may allow resources, that are provisioned in the control plane VCN **216** that is contained in the service tenancy **219**, to be deployed or otherwise used in the data plane VCN **218** that is contained in the customer tenancy **221**.

[0063] In other examples, the customer of the IaaS provider may have databases that live in the customer tenancy **221**. In this example, the control plane VCN **216** can include the data plane mirror app tier **240** that can include app subnet(s) **226**. The data plane mirror app tier **240** can reside in the data plane VCN **218**, but the data plane mirror app tier **240** may not live in the data plane VCN **218**. That is, the data plane mirror app tier **240** may have access to the customer tenancy **221**, but the data plane mirror app tier **240** may not exist in the data plane VCN **218** or be owned or operated by the customer of the IaaS provider. The data plane mirror app tier **240** may be configured to make calls to the data plane VCN **218** but may not be configured to make calls to any

entity contained in the control plane VCN **216**. The customer may desire to deploy or otherwise use resources in the data plane VCN **218** that are provisioned in the control plane VCN **216**, and the data plane mirror app tier **240** can facilitate the desired deployment, or other usage of resources, of the customer.

[0064] In some embodiments, the customer of the IaaS provider can apply filters to the data plane VCN **218**. In this embodiment, the customer can determine what the data plane VCN **218** can access, and the customer may restrict access to public Internet **254** from the data plane VCN **218**. The IaaS provider may not be able to apply filters or otherwise control access of the data plane VCN **218** to any outside networks or databases. Applying filters and controls by the customer onto the data plane VCN **218**, contained in the customer tenancy **221**, can help isolate the data plane VCN **218** from other customers and from public Internet **254**.

[0065] In some embodiments, cloud services **256** can be called by the service gateway **236** to access services that may not exist on public Internet **254**, on the control plane VCN **216**, or on the data plane VCN **218**. The connection between cloud services **256** and the control plane VCN **216** or the data plane VCN **218** may not be live or continuous. Cloud services **256** may exist on a different network owned or operated by the IaaS provider. Cloud services **256** may be configured to receive calls from the service gateway **236** and may be configured to not receive calls from public Internet **254**. Some cloud services **256** may be isolated from other cloud services **256**, and the control plane VCN **216** may be isolated from cloud services **256** that may not be in the same region as the control plane VCN **216**. For example, the control plane VCN **216** may be located in “Region 1,” and cloud service “Deployment 1,” may be located in Region 1 and in “Region 2.” If a call to Deployment 1 is made by the service gateway **236** contained in the control plane VCN **216** located in Region 1, the call may be transmitted to Deployment 1 in Region 1. In this example, the control plane VCN **216**, or Deployment 1 in Region 1, may not be communicatively coupled to, or otherwise in communication with, Deployment 1 in Region 2.

[0066] FIG. 3 is a block diagram **300** illustrating another example pattern of an IaaS architecture, according to at least one embodiment. Service operators **302** (e.g., service operators **102** of FIG. 1) can be communicatively coupled to a secure host tenancy **304** (e.g., the secure host tenancy **104** of FIG. 1) that can include a virtual cloud network (VCN) **306** (e.g., the VCN **106** of FIG. 1) and a secure host subnet **308** (e.g., the secure host subnet **108** of FIG. 1). The VCN **306** can include an LPG **310** (e.g., the LPG **110** of FIG. 1) that can be communicatively coupled to an SSH VCN **312** (e.g., the SSH VCN **112** of FIG. 1) via an LPG **310** contained in the SSH VCN **312**. The SSH VCN **312** can include an SSH subnet **314** (e.g., the SSH subnet **114** of FIG. 1), and the SSH VCN **312** can be communicatively coupled to a control plane VCN **316** (e.g., the control plane VCN **116** of FIG. 1) via an LPG **310** contained in the control plane VCN **316** and to a data plane VCN **318** (e.g., the data plane **118** of FIG. 1) via an LPG **310** contained in the data plane VCN **318**. The control plane VCN **316** and the data plane VCN **318** can be contained in a service tenancy **319** (e.g., the service tenancy **119** of FIG. 1).

[0067] The control plane VCN **316** can include a control plane DMZ tier **320** (e.g., the control plane DMZ tier **120** of

FIG. 1) that can include load balancer (LB) subnet(s) 322 (e.g., LB subnet(s) 122 of FIG. 1), a control plane app tier 324 (e.g., the control plane app tier 124 of FIG. 1) that can include app subnet(s) 326 (e.g., similar to app subnet(s) 126 of FIG. 1), a control plane data tier 328 (e.g., the control plane data tier 128 of FIG. 1) that can include DB subnet(s) 330. The LB subnet(s) 322 contained in the control plane DMZ tier 320 can be communicatively coupled to the app subnet(s) 326 contained in the control plane app tier 324 and to an Internet gateway 334 (e.g., the Internet gateway 134 of FIG. 1) that can be contained in the control plane VCN 316, and the app subnet(s) 326 can be communicatively coupled to the DB subnet(s) 330 contained in the control plane data tier 328 and to a service gateway 336 (e.g., the service gateway of FIG. 1) and a network address translation (NAT) gateway 338 (e.g., the NAT gateway 138 of FIG. 1). The control plane VCN 316 can include the service gateway 336 and the NAT gateway 338.

[0068] The data plane VCN 318 can include a data plane app tier 346 (e.g., the data plane app tier 146 of FIG. 1), a data plane DMZ tier 348 (e.g., the data plane DMZ tier 148 of FIG. 1), and a data plane data tier 350 (e.g., the data plane data tier 150 of FIG. 1). The data plane DMZ tier 348 can include LB subnet(s) 322 that can be communicatively coupled to trusted app subnet(s) 360 and untrusted app subnet(s) 362 of the data plane app tier 346 and the Internet gateway 334 contained in the data plane VCN 318. The trusted app subnet(s) 360 can be communicatively coupled to the service gateway 336 contained in the data plane VCN 318, the NAT gateway 338 contained in the data plane VCN 318, and DB subnet(s) 330 contained in the data plane data tier 350. The untrusted app subnet(s) 362 can be communicatively coupled to the service gateway 336 contained in the data plane VCN 318 and DB subnet(s) 330 contained in the data plane data tier 350. The data plane data tier 350 can include DB subnet(s) 330 that can be communicatively coupled to the service gateway 336 contained in the data plane VCN 318.

[0069] The untrusted app subnet(s) 362 can include one or more primary VNICs 364(1)-(N) that can be communicatively coupled to tenant virtual machines (VMs) 366(1)-(N). Each tenant VM 366(1)-(N) can be communicatively coupled to a respective app subnet 367(1)-(N) that can be contained in respective container egress VCNs 368(1)-(N) that can be contained in respective customer tenancies 370(1)-(N). Respective secondary VNICs 372(1)-(N) can facilitate communication between the untrusted app subnet(s) 362 contained in the data plane VCN 318 and the app subnet contained in the container egress VCNs 368(1)-(N). Each container egress VCNs 368(1)-(N) can include a NAT gateway 338 that can be communicatively coupled to public Internet 354 (e.g., public Internet 154 of FIG. 1).

[0070] The Internet gateway 334 contained in the control plane VCN 316 and contained in the data plane VCN 318 can be communicatively coupled to a metadata management service 352 (e.g., the metadata management system 152 of FIG. 1) that can be communicatively coupled to public Internet 354. Public Internet 354 can be communicatively coupled to the NAT gateway 338 contained in the control plane VCN 316 and contained in the data plane VCN 318. The service gateway 336 contained in the control plane VCN 316 and contained in the data plane VCN 318 can be communicatively coupled to cloud services 356.

[0071] In some embodiments, the data plane VCN 318 can be integrated with customer tenancies 370. This integration can be useful or desirable for customers of the IaaS provider in some cases such as a case that may desire support when executing code. The customer may provide code to run that may be destructive, may communicate with other customer resources, or may otherwise cause undesirable effects. In response to this, the IaaS provider may determine whether to run code given to the IaaS provider by the customer.

[0072] In some examples, the customer of the IaaS provider may grant temporary network access to the IaaS provider and request a function to be attached to the data plane app tier 346. Code to run the function may be executed in the VMs 366(1)-(N), and the code may not be configured to run anywhere else on the data plane VCN 318. Each VM 366(1)-(N) may be connected to one customer tenancy 370. Respective containers 371(1)-(N) contained in the VMs 366(1)-(N) may be configured to run the code. In this case, there can be a dual isolation (e.g., the containers 371(1)-(N) running code, where the containers 371(1)-(N) may be contained in at least the VM 366(1)-(N) that are contained in the untrusted app subnet(s) 362), which may help prevent incorrect or otherwise undesirable code from damaging the network of the IaaS provider or from damaging a network of a different customer. The containers 371(1)-(N) may be communicatively coupled to the customer tenancy 370 and may be configured to transmit or receive data from the customer tenancy 370. The containers 371(1)-(N) may not be configured to transmit or receive data from any other entity in the data plane VCN 318. Upon completion of running the code, the IaaS provider may kill or otherwise dispose of the containers 371(1)-(N).

[0073] In some embodiments, the trusted app subnet(s) 360 may run code that may be owned or operated by the IaaS provider. In this embodiment, the trusted app subnet(s) 360 may be communicatively coupled to the DB subnet(s) 330 and be configured to execute CRUD operations in the DB subnet(s) 330. The untrusted app subnet(s) 362 may be communicatively coupled to the DB subnet(s) 330, but in this embodiment, the untrusted app subnet(s) may be configured to execute read operations in the DB subnet(s) 330. The containers 371(1)-(N) that can be contained in the VM 366(1)-(N) of each customer and that may run code from the customer may not be communicatively coupled with the DB subnet(s) 330.

[0074] In other embodiments, the control plane VCN 316 and the data plane VCN 318 may not be directly communicatively coupled. In this embodiment, there may be no direct communication between the control plane VCN 316 and the data plane VCN 318. However, communication can occur indirectly through at least one method. An LPG 310 may be established by the IaaS provider that can facilitate communication between the control plane VCN 316 and the data plane VCN 318. In another example, the control plane VCN 316 or the data plane VCN 318 can make a call to cloud services 356 via the service gateway 336. For example, a call to cloud services 356 from the control plane VCN 316 can include a request for a service that can communicate with the data plane VCN 318.

[0075] FIG. 4 is a block diagram 400 illustrating another example pattern of an IaaS architecture, according to at least one embodiment. Service operators 402 (e.g., service operators 102 of FIG. 1) can be communicatively coupled to a secure host tenancy 404 (e.g., the secure host tenancy 104 of

FIG. 1) that can include a virtual cloud network (VCN) 406 (e.g., the VCN 106 of FIG. 1) and a secure host subnet 408 (e.g., the secure host subnet 108 of FIG. 1). The VCN 406 can include an LPG 410 (e.g., the LPG 110 of FIG. 1) that can be communicatively coupled to an SSH VCN 412 (e.g., the SSH VCN 112 of FIG. 1) via an LPG 410 contained in the SSH VCN 412. The SSH VCN 412 can include an SSH subnet 414 (e.g., the SSH subnet 114 of FIG. 1), and the SSH VCN 412 can be communicatively coupled to a control plane VCN 416 (e.g., the control plane VCN 116 of FIG. 1) via an LPG 410 contained in the control plane VCN 416 and to a data plane VCN 418 (e.g., the data plane 118 of FIG. 1) via an LPG 410 contained in the data plane VCN 418. The control plane VCN 416 and the data plane VCN 418 can be contained in a service tenancy 419 (e.g., the service tenancy 119 of FIG. 1).

[0076] The control plane VCN 416 can include a control plane DMZ tier 420 (e.g., the control plane DMZ tier 120 of FIG. 1) that can include LB subnet(s) 422 (e.g., LB subnet(s) 122 of FIG. 1), a control plane app tier 424 (e.g., the control plane app tier 124 of FIG. 1) that can include app subnet(s) 426 (e.g., app subnet(s) 126 of FIG. 1), a control plane data tier 428 (e.g., the control plane data tier 128 of FIG. 1) that can include DB subnet(s) 430 (e.g., DB subnet(s) 330 of FIG. 3). The LB subnet(s) 422 contained in the control plane DMZ tier 420 can be communicatively coupled to the app subnet(s) 426 contained in the control plane app tier 424 and to an Internet gateway 434 (e.g., the Internet gateway 134 of FIG. 1) that can be contained in the control plane VCN 416, and the app subnet(s) 426 can be communicatively coupled to the DB subnet(s) 430 contained in the control plane data tier 428 and to a service gateway 436 (e.g., the service gateway of FIG. 1) and a network address translation (NAT) gateway 438 (e.g., the NAT gateway 138 of FIG. 1). The control plane VCN 416 can include the service gateway 436 and the NAT gateway 438.

[0077] The data plane VCN 418 can include a data plane app tier 446 (e.g., the data plane app tier 146 of FIG. 1), a data plane DMZ tier 448 (e.g., the data plane DMZ tier 148 of FIG. 1), and a data plane data tier 450 (e.g., the data plane data tier 150 of FIG. 1). The data plane DMZ tier 448 can include LB subnet(s) 422 that can be communicatively coupled to trusted app subnet(s) 460 (e.g., trusted app subnet(s) 360 of FIG. 3) and untrusted app subnet(s) 462 (e.g., untrusted app subnet(s) 362 of FIG. 3) of the data plane app tier 446 and the Internet gateway 434 contained in the data plane VCN 418. The trusted app subnet(s) 460 can be communicatively coupled to the service gateway 436 contained in the data plane VCN 418, the NAT gateway 438 contained in the data plane VCN 418, and DB subnet(s) 430 contained in the data plane data tier 450. The untrusted app subnet(s) 462 can be communicatively coupled to the service gateway 436 contained in the data plane VCN 418 and DB subnet(s) 430 contained in the data plane data tier 450. The data plane data tier 450 can include DB subnet(s) 430 that can be communicatively coupled to the service gateway 436 contained in the data plane VCN 418.

[0078] The untrusted app subnet(s) 462 can include primary VNICS 464(1)-(N) that can be communicatively coupled to tenant virtual machines (VMs) 466(1)-(N) residing within the untrusted app subnet(s) 462. Each tenant VM 466(1)-(N) can run code in a respective container 467(1)-(N), and be communicatively coupled to an app subnet 426 that can be contained in a data plane app tier 446 that can be

contained in a container egress VCN 468. Respective secondary VNICS 472(1)-(N) can facilitate communication between the untrusted app subnet(s) 462 contained in the data plane VCN 418 and the app subnet contained in the container egress VCN 468. The container egress VCN can include a NAT gateway 438 that can be communicatively coupled to public Internet 454 (e.g., public Internet 154 of FIG. 1).

[0079] The Internet gateway 434 contained in the control plane VCN 416 and contained in the data plane VCN 418 can be communicatively coupled to a metadata management service 452 (e.g., the metadata management system 152 of FIG. 1) that can be communicatively coupled to public Internet 454. Public Internet 454 can be communicatively coupled to the NAT gateway 438 contained in the control plane VCN 416 and contained in the data plane VCN 418. The service gateway 436 contained in the control plane VCN 416 and contained in the data plane VCN 418 can be communicatively coupled to cloud services 456.

[0080] In some examples, the pattern illustrated by the architecture of block diagram 400 of FIG. 4 may be considered an exception to the pattern illustrated by the architecture of block diagram 300 of FIG. 3 and may be desirable for a customer of the IaaS provider if the IaaS provider cannot directly communicate with the customer (e.g., a disconnected region). The respective containers 467(1)-(N) that are contained in the VMs 466(1)-(N) for each customer can be accessed in real-time by the customer. The containers 467(1)-(N) may be configured to make calls to respective secondary VNICS 472(1)-(N) contained in app subnet(s) 426 of the data plane app tier 446 that can be contained in the container egress VCN 468. The secondary VNICS 472(1)-(N) can transmit the calls to the NAT gateway 438 that may transmit the calls to public Internet 454. In this example, the containers 467(1)-(N) that can be accessed in real-time by the customer can be isolated from the control plane VCN 416 and can be isolated from other entities contained in the data plane VCN 418. The containers 467(1)-(N) may also be isolated from resources from other customers.

[0081] In other examples, the customer can use the containers 467(1)-(N) to call cloud services 456. In this example, the customer may run code in the containers 467(1)-(N) that requests a service from cloud services 456. The containers 467(1)-(N) can transmit this request to the secondary VNICS 472(1)-(N) that can transmit the request to the NAT gateway that can transmit the request to public Internet 454. Public Internet 454 can transmit the request to LB subnet(s) 422 contained in the control plane VCN 416 via the Internet gateway 434. In response to determining the request is valid, the LB subnet(s) can transmit the request to app subnet(s) 426 that can transmit the request to cloud services 456 via the service gateway 436.

[0082] It should be appreciated that IaaS architectures 100, 200, 300, 400 depicted in the figures may have other components than those depicted. Further, the embodiments shown in the figures are only some examples of a cloud infrastructure system that may incorporate an embodiment of the disclosure. In some other embodiments, the IaaS systems may have more or fewer components than shown in the figures, may combine two or more components, or may have a different configuration or arrangement of components.

[0083] In certain embodiments, the IaaS systems described herein may include a suite of applications, middle-

ware, and database service offerings that are delivered to a customer in a self-service, subscription-based, elastically scalable, reliable, highly available, and secure manner. An example of such an IaaS system is the Oracle Cloud Infrastructure (OCI) provided by the present assignee.

Computer System

[0084] FIG. 5 illustrates an example computer system **500**, in which various embodiments described herein may be implemented. The system **500** may be used to implement any of the computer systems described above. As shown in the figure, computer system **500** includes a processing unit **504** that communicates with a number of peripheral subsystems via a bus subsystem **502**. These peripheral subsystems may include a processing acceleration unit **506**, an I/O subsystem **508**, a storage subsystem **518** and a communications subsystem **524**. Storage subsystem **518** includes tangible computer-readable storage media **522** and a system memory **510**.

[0085] Bus subsystem **502** provides a mechanism for letting the various components and subsystems of computer system **500** communicate with each other as intended. Although bus subsystem **502** is shown schematically as a single bus, alternative embodiments of the bus subsystem may utilize multiple buses. Bus subsystem **502** may be any of several types of bus structures including a memory bus or memory controller, a peripheral bus, and a local bus using any of a variety of bus architectures. For example, such architectures may include an Industry Standard Architecture (ISA) bus, Micro Channel Architecture (MCA) bus, Enhanced ISA (EISA) bus, Video Electronics Standards Association (VESA) local bus, and Peripheral Component Interconnect (PCI) bus, which can be implemented as a Mezzanine bus manufactured to the IEEE P1386.1 standard.

[0086] Processing unit **504**, which can be implemented as one or more integrated circuits (e.g., a conventional microprocessor or microcontroller), controls the operation of computer system **500**. One or more processors may be included in processing unit **504**. These processors may include single core or multicore processors. In certain embodiments, processing unit **504** may be implemented as one or more independent processing units **532** and/or **534** with single or multicore processors included in each processing unit. In other embodiments, processing unit **504** may also be implemented as a quad-core processing unit formed by integrating two dual-core processors into a single chip.

[0087] In various embodiments, processing unit **504** can execute a variety of programs in response to program code and can maintain multiple concurrently executing programs or processes. At any given time, some or all of the program code to be executed can be resident in processor(s) **504** and/or in storage subsystem **518**. Through suitable programming, processor(s) **504** can provide various functionalities described above. Computer system **500** may additionally include a processing acceleration unit **506**, which can include a digital signal processor (DSP), a special-purpose processor, and/or the like.

[0088] I/O subsystem **508** may include user interface input devices and user interface output devices. User interface input devices may include a keyboard, pointing devices such as a mouse or trackball, a touchpad or touch screen incorporated into a display, a scroll wheel, a click wheel, a dial, a button, a switch, a keypad, audio input devices with voice command recognition systems, microphones, and other

types of input devices. User interface input devices may include, for example, motion sensing and/or gesture recognition devices such as the Microsoft Kinect® motion sensor that enables users to control and interact with an input device, such as the Microsoft Xbox® 360 game controller, through a natural user interface using gestures and spoken commands. User interface input devices may also include eye gesture recognition devices such as the Google Glass® blink detector that detects eye activity (e.g., ‘blinking’ while taking pictures and/or making a menu selection) from users and transforms the eye gestures as input into an input device (e.g., Google Glass®). Additionally, user interface input devices may include voice recognition sensing devices that enable users to interact with voice recognition systems (e.g., Siri® navigator), through voice commands.

[0089] User interface input devices may also include, without limitation, three dimensional (3D) mice, joysticks or pointing sticks, gamepads and graphic tablets, and audio/visual devices such as speakers, digital cameras, digital camcorders, portable media players, webcams, image scanners, fingerprint scanners, barcode reader 3D scanners, 3D printers, laser rangefinders, and eye gaze tracking devices. Additionally, user interface input devices may include, for example, medical imaging input devices such as computed tomography, magnetic resonance imaging, position emission tomography, medical ultrasonography devices. User interface input devices may also include, for example, audio input devices such as MIDI keyboards, digital musical instruments and the like.

[0090] User interface output devices may include a display subsystem, indicator lights, or non-visual displays such as audio output devices, etc. The display subsystem may be a cathode ray tube (CRT), a flat-panel device, such as that using a liquid crystal display (LCD) or plasma display, a projection device, a touch screen, and the like. In general, use of the term “output device” is intended to include all possible types of devices and mechanisms for outputting information from computer system **500** to a user or other computer. For example, user interface output devices may include, without limitation, a variety of display devices that visually convey text, graphics and audio/video information such as monitors, printers, speakers, headphones, automotive navigation systems, plotters, voice output devices, and modems.

[0091] Computer system **500** may comprise a storage subsystem **518** that provides a tangible non-transitory computer-readable storage medium for storing software and data constructs that provide the functionality of the embodiments described in this disclosure. The software can include programs, code modules, instructions, scripts, etc., that when executed by one or more cores or processors of processing unit **504** provide the functionality described above. Storage subsystem **518** may also provide a repository for storing data used in accordance with the present disclosure.

[0092] As depicted in the example in FIG. 5, storage subsystem **518** can include various components including a system memory **510**, computer-readable storage media **522**, and a computer readable storage media reader **520**. System memory **510** may store program instructions that are loadable and executable by processing unit **504**. System memory **510** may also store data that is used during the execution of the instructions and/or data that is generated during the execution of the program instructions. Various different kinds of programs may be loaded into system memory **510**

including but not limited to client applications, Web browsers, mid-tier applications, relational database management systems (RDBMS), virtual machines, containers, etc.

[0093] System memory **510** may also store an operating system **516**. Examples of operating system **516** may include various versions of Microsoft Windows®, Apple Macintosh®, and/or Linux operating systems, a variety of commercially-available UNIX® or UNIX-like operating systems (including without limitation the variety of GNU/Linux operating systems, the Google Chrome® OS, and the like) and/or mobile operating systems such as iOS, Windows® Phone, Android® OS, BlackBerry® OS, and Palm® OS operating systems. In certain implementations where computer system **500** executes one or more virtual machines, the virtual machines along with their guest operating systems (GOSs) may be loaded into system memory **510** and executed by one or more processors or cores of processing unit **504**.

[0094] System memory **510** can come in different configurations depending upon the type of computer system **500**. For example, system memory **510** may be volatile memory (such as random access memory (RAM)) and/or non-volatile memory (such as read-only memory (ROM), flash memory, etc.) Different types of RAM configurations may be provided including a static random access memory (SRAM), a dynamic random access memory (DRAM), and others. In some implementations, system memory **510** may include a basic input/output system (BIOS) containing basic routines that help to transfer information between elements within computer system **500**, such as during start-up.

[0095] Computer-readable storage media **522** may represent remote, local, fixed, and/or removable storage devices plus storage media for temporarily and/or more permanently containing, storing, computer-readable information for use by computer system **500** including instructions executable by processing unit **504** of computer system **500**.

[0096] Computer-readable storage media **522** can include any appropriate media known or used in the art, including storage media and communication media, such as but not limited to, volatile and non-volatile, removable and non-removable media implemented in any method or technology for storage and/or transmission of information. This can include tangible computer-readable storage media such as RAM, ROM, electronically erasable programmable ROM (EEPROM), flash memory or other memory technology, CD-ROM, digital versatile disk (DVD), or other optical storage, magnetic cassettes, magnetic tape, magnetic disk storage or other magnetic storage devices, or other tangible computer readable media.

[0097] By way of example, computer-readable storage media **522** may include a hard disk drive that reads from or writes to non-removable, nonvolatile magnetic media, a magnetic disk drive that reads from or writes to a removable, nonvolatile magnetic disk, and an optical disk drive that reads from or writes to a removable, nonvolatile optical disk such as a CD ROM, DVD, and Blu-Ray® disk, or other optical media. Computer-readable storage media **522** may include, but is not limited to, Zip® drives, flash memory cards, universal serial bus (USB) flash drives, secure digital (SD) cards, DVD disks, digital video tape, and the like. Computer-readable storage media **522** may also include, solid-state drives (SSD) based on non-volatile memory such as flash-memory based SSDs, enterprise flash drives, solid state ROM, and the like, SSDs based on volatile memory

such as solid state RAM, dynamic RAM, static RAM, DRAM-based SSDs, magnetoresistive RAM (MRAM) SSDs, and hybrid SSDs that use a combination of DRAM and flash memory based SSDs. The disk drives and their associated computer-readable media may provide non-volatile storage of computer-readable instructions, data structures, program modules, and other data for computer system **500**.

[0098] Machine-readable instructions executable by one or more processors or cores of processing unit **504** may be stored on a non-transitory computer-readable storage medium. A non-transitory computer-readable storage medium can include physically tangible memory or storage devices that include volatile memory storage devices and/or non-volatile storage devices. Examples of non-transitory computer-readable storage medium include magnetic storage media (e.g., disk or tapes), optical storage media (e.g., DVDs, CDs), various types of RAM, ROM, or flash memory, hard drives, floppy drives, detachable memory drives (e.g., USB drives), or other type of storage device.

[0099] Communications subsystem **524** provides an interface to other computer systems and networks. Communications subsystem **524** serves as an interface for receiving data from and transmitting data to other systems from computer system **500**. For example, communications subsystem **524** may enable computer system **500** to connect to one or more devices via the Internet. In some embodiments communications subsystem **524** can include radio frequency (RF) transceiver components for accessing wireless voice and/or data networks (e.g., using cellular telephone technology, advanced data network technology, such as 3G, 4G or EDGE (enhanced data rates for global evolution), WiFi (IEEE 802.11 family standards, or other mobile communication technologies, or any combination thereof), global positioning system (GPS) receiver components, and/or other components. In some embodiments communications subsystem **524** can provide wired network connectivity (e.g., Ethernet) in addition to or instead of a wireless interface.

[0100] In some embodiments, communications subsystem **524** may also receive input communication in the form of structured and/or unstructured data feeds **526**, event streams **528**, event updates **530**, and the like on behalf of one or more users who may use computer system **500**.

[0101] By way of example, communications subsystem **524** may be configured to receive data feeds **526** in real-time from users of social networks and/or other communication services such as Twitter® feeds, Facebook® updates, web feeds such as Rich Site Summary (RSS) feeds, and/or real-time updates from one or more third party information sources.

[0102] Additionally, communications subsystem **524** may also be configured to receive data in the form of continuous data streams, which may include event streams **528** of real-time events and/or event updates **530**, that may be continuous or unbounded in nature with no explicit end. Examples of applications that generate continuous data may include, for example, sensor data applications, financial tickers, network performance measuring tools (e.g., network monitoring and traffic management applications), click-stream analysis tools, automobile traffic monitoring, and the like.

[0103] Communications subsystem **524** may also be configured to output the structured and/or unstructured data feeds **526**, event streams **528**, event updates **530**, and the like

to one or more databases that may be in communication with one or more streaming data source computers coupled to computer system 500.

[0104] Computer system 500 can be one of various types, including a handheld portable device (e.g., an iPhone® cellular phone, an iPad® computing tablet, a PDA), a wearable device (e.g., a Google Glass® head mounted display), a PC, a workstation, a mainframe, a kiosk, a server rack, or any other data processing system.

[0105] Due to the ever-changing nature of computers and networks, the description of computer system 500 depicted in the figure is intended only as a specific example. Many other configurations having more or fewer components than the system depicted in the figure are possible. For example, customized hardware might also be used and/or particular elements might be implemented in hardware, firmware, software (including applets), or a combination. Further, connection to other computing devices, such as network input/output devices, may be employed. Based on the disclosure and teachings provided herein, a person of ordinary skill in the art will appreciate other ways and/or methods to implement the various embodiments.

AI Training Using Accessibility Data

[0106] As described above, an end-user (also referred to herein as a “user”) may rely on a virtual assistant service, otherwise referred to as an assistive AI model or service, to perform tasks. FIG. 6 is a block diagram illustrating a system architecture 600 for training an AI model to perform assistive business functions, or tasks, according to at least one embodiment.

[0107] A user may access the assistive functionality through a user device, such as the user device 604 illustrated in FIG. 6. The user device 604 may be substantially similar to the computer system 500 described above with respect to FIG. 5. For example, the user device 604 may include a processing unit substantially similar to the processing unit 504, a processing acceleration unit substantially similar to the processing acceleration unit 506, a I/O subsystem substantially similar to the I/O subsystem 508, a storage subsystem substantially similar to the storage subsystem 518, and/or a communications subsystem substantially similar to the communications subsystem 524.

[0108] The user device 604 is provided with an assistant agent 608 (software stored on the user device 604 and executed by a processing unit of the user device 604, such as, for example, as a background application) configured to receive accessibility data from an accessibility data source 612. The assistant agent 608 receives the accessibility data by making one or more calls via an accessibility application programming interface (API), otherwise referred to herein as an accessibility interface, to request the accessibility data. Calls made via the accessibility API are calls that conform to the accessibility API, which may conform to an accessibility standard. The assistant agent 608 may periodically request the accessibility data (e.g., at predetermined intervals) or request the accessibility data responsive to receiving a prompt from a user to complete a task.

[0109] The accessibility data source 612 is a platform that renders a user interface (“UI”) displayed by the user device 604. For example, the accessibility data source 612 may include an operating system (“OS”) executing on the user device 604, a browser application executing on the user device 604, and/or another software application executing

on the user device 604 (e.g., software used for business functions). Accordingly, the accessibility data is generated by the platform executing on the user device 604 during rendering of the UI.

[0110] The accessibility data, which is associated with the UI displayed by the user device 604, includes information identifying UI elements in the UI of the user device 604 and/or information identifying UI events in the UI of the user device 604. UI elements may include, for example, windows, buttons, scroll bars, menus, and/or the like. UI events include changes in a state or condition of the UI elements. UI events may be caused by user interaction with the UI elements, such as keyboard strokes, mouse clicks, software execution, and/or the like. For example, keyboard strokes may cause text rendered in a UI element to change.

[0111] In addition to being received by the assistant agent 608, the accessibility data may be received by an application configured to provide accessibility functionality. The accessibility functionality may be, for example, functionality beyond that which is offered by the UI rendered on the user device 604 to facilitate user interaction with the UI (e.g., by users with disabilities). Accessibility functionality may include, but is not limited to, screen magnifiers, screen readers, text-to-speech software, speech recognition software, and/or the like.

[0112] The accessibility data (i.e., the information identifying UI elements and/or the information identifying UI events in the UI of the user device 604) is generated based on metadata exposed via the accessibility API. Metadata exposed via the accessibility API is metadata that conforms to the accessibility API, which may conform to a standard, such as an accessibility or UI standard. Metadata generally refers to labels created by, for example, a developer to identify characteristics of UI elements.

[0113] In some instances, the information identifying the UI elements in a UI of the user device 604 is organized as a hierarchical tree. For example, FIGS. 7A and 7B illustrate example accessibility data. FIG. 7A illustrates an example accessibility data tree 700 of accessibility data associated with UI elements of an example UI of user device 604 (i.e., any one of the first user device 604a or the second user device 604b). Each hierarchical level of the hierarchical tree 700 includes at least one node respectively representing a UI element of the example UI and associated metadata for each element. The hierarchical relationships between respective UI elements of the example UI may be specified by the metadata.

[0114] For each respective UI element, the metadata may also specify a respective element type of the respective UI element and/or a respective identifier of the respective UI element. For example, as indicated by the hierarchical tree 700, a UI element of the example UI 702 includes a pane-type UI element 704 identified as “Desktop.”

[0115] Each respective UI element may include one or more secondary UI elements. For example, the pane-type UI element may include a first window-type UI element 708 identified as “Window1” and a second window-type UI element 710 identified as “Window2” that are each at a hierarchical level of the hierarchical tree 700 below the pane-type UI element 704. In other words, each hierarchical level of the hierarchical tree 700 includes at least one node respectively representing a UI element.

[0116] As further illustrated in FIG. 7A, the first window-type UI element 708 includes, at a lower hierarchical level

than the pane-type UI element **708**, a TitleBar-type UI element **712** identified as “TitleBar1,” and defining a title bar portion of the window-type UI element **708**, a menu-type UI element **716** identified as “Menu1” and defining a menu of the window-type UI element **708**, a StatusBar-type UI element **720** identified as “StatusBar1” and defining a status bar of the window-type UI element **708**, and a group-type UI element **724** identified as “Group1,” and defining a UI group included in the window-type UI element **708**. The group-type UI element **724** further includes a text-type UI element **728** identified as “Group1,” defining text displayed within the group-type UI element **724**, and a tree-type UI element **732** displayed within the group-type UI element **724**.

[0117] For each respective UI element, the metadata may also specify a respective state or condition of the UI element. For example, as illustrated in FIG. 7B, the accessibility data (e.g., the metadata) may also provide additional metadata of a particular UI element (e.g., the tree-type UI element **732** of FIG. 7A). As illustrated in FIG. 7B, additional metadata **740** defining states or conditions of a given UI element may include other identifiers (e.g., an automation identifier, a name, a class name), control information (e.g., a control type, a localized control type), framework information (e.g., a framework type, a framework identifier), process information (e.g., a process identifier), location information of the UI element within the example UI (e.g., coordinates and/or dimensions of the UI element), pattern information of the UI element, and/or the like.

[0118] Conditions and/or states of a respective UI element may change, for example, responsive to user interaction with the UI element or with other UI elements, responsive to software execution, and/or the like. For example, responsive to a user interacting with a respective UI element to resize the UI element, bounding box information associated with the UI element may change (e.g., pixel coordinates and/or dimensions of the UI element). As described above, UI events include changes in a state or condition of the UI elements. The information identifying UI events (i.e., information included as accessibility data requested by, for example, the assistant agent **608**) may include notifications of such changes in states or conditions of the UI elements.

[0119] The metadata associated with the UI elements and/or UI events may be specified in application code of application for which the example UI is being rendered and/or content code of web content for which the example UI is being rendered.

[0120] Referring again to FIG. 6, the assistant agent **608** provides the accessibility data received from the accessibility data source **612** to the artificial intelligence (“AI”) model **616** as training data. As described in further detail below, the assistant agent **608** may provide the accessibility data with additional information regarding the operating context of the user device **604**, such as, for example, what applications are running, hardware events, what tabs are open, etc. (e.g., to the extent such information is provided as part of the accessibility information). While illustrated herein for simplicity as residing on the user device **604**, in some instances, the AI model **616** resides external to the user device **604** in a device communicatively connected to the user device **604** (e.g., a local or remote server). Accordingly, in some instances, functionality provided by the AI model is included as part of the cloud services **456** described above with respect to FIG. 4.

[0121] The AI model may be, for example, a large language model (“LLM”) that is fine-tuned and/or prompt-tuned according to a curated set of tasks and context surrounding those tasks, including, for example, the accessibility data provided via the assistant agent **608**. The LLM may be a suitably large model having a minimum number of parameters (e.g., at least 7 billion parameters, at least 25 billion parameters, at least 50 billion parameters, etc.). To fine-tune a pre-trained model, a training data set including the information noted above can be provided to the model (e.g., by defining a path to such training data, which may be stored in one or more object storage buckets on a cloud platform).

[0122] The AI model **616** uses the received accessibility data to learn user work patterns and task completion behavior. For example, FIG. 8 illustrates a first example workflow **800** for training the AI model **616**. As illustrated in FIG. 8, based on the received accessibility data, the assistant agent **608** identifies a window (i.e., a window UI element) displayed on a user interface of the user device **604**, the window having one or more UI elements (e.g., buttons, text boxes, scroll bars, menus, etc.) (at block **804**).

[0123] Using the accessibility data, the assistant agent **608** identifies the UI elements within the window (at block **808**), and determines, using the accessibility data, placement (e.g., coordinates and/or dimensions) of the identified UI elements within the window and the placement of the window within a windowing system (e.g., a pane) (at block **812**).

[0124] Using the accessibility data, the assistant agent **608** determines one or more UI events associated with the window (at block **816**). As described above, UI events may be caused by, for example, keyboard strokes, mouse clicks, and/or the like. The assistant agent **608** may repeat the steps of blocks **804-816** for each window displayed by the user device **604** (at block **820**).

[0125] The assistant agent **608** provides the information identifying the UI elements in each window and the information identifying the UI events associated with each window (e.g., the information identified in blocks **808-816**) to the AI model **616** to train the AI model **616** to identify user actions, behavior patterns, work completion patterns, and/or the like (at block **824**). The AI model **616** may be trained, using the accessibility data, to identify tasks performed by a user of the user device **604**, times of day that each task is performed, frequency at which each task is performed, the context surrounding a particular task (e.g., an action that triggers performance of a task), and/or the like.

[0126] As part of training the AI model **616**, a task or action is broken down into multiple smaller operations, that define the action, and infers the intended result of each user action (at block **828**). As one non-limiting example, the AI model **616** may be trained to identify a set of operations that are performed to complete a particular task, such as, for example, processing of a new purchase order. The set of operations may include receiving an email identifying a new purchase order (e.g., a first purchase order identifying a first item for purchase, a first quantity, a first manufacturer, and/or the like), navigation to a spreadsheet or other database or data file or application containing price information, selection of the spreadsheet associated with a price of the first item identified by the purchase order, copying of the selected data from the selected cell, opening a calculator application on the user device **604**, entering or pasting the price data into the calculator application, multiplying the

price by the corresponding first quantity identified in the purchase order, and typing or otherwise entering the result of the calculation in another application (e.g., an email addressed to the manufacturer, an application or webpage for expensing or completing purchases, etc.). Accordingly, AI model 616 is trained to determine a set of operations defining an action having an intended result of, in the example described above, completing a purchase order identified in an email.

[0127] Once trained, the assistant agent 608 uses the trained AI model 616 to determine a recommended action to preemptively complete tasks by collecting operations (through accessibility API calls) performed on window elements using the learned set of operations for solving a particular task (at block 832). Referring to the example scenario described above, in response to determining that the user received a purchase order in an email (per accessibility data received via the assistant agent 608 and input to the model 616, the model 616 may output a recommended action to complete the purchase order, wherein the recommended action is defined by the set operations required to perform the recommended action. Similarly, in other implementations, once trained, a user may input a prompt and query to the AI model 616 to complete a task and the AI model 616 may output a recommended action (e.g., to the assistant agent 608, configured to implement or coordinate execution of the recommended action) that defines a set of operations required to complete the task. In other words, the trained AI model 616 may be used to as a user operates the user device 604 to predict tasks that may need completion (e.g., by processing accessibility data, which may also be used to as further training data) and/or the trained AI model 616 may be used as a virtual assistant configured to respond to user prompts and queries.

[0128] In some instances, the assistant agent 608 outputs a command to perform the recommended action (e.g., on the user device 604) by making a call via the accessibility API that causes one or more results of the recommended action to render on the UI of the user device 604 (e.g., cause an email draft to populate with resulting calculated information, cause values in an application such as a database to update, cause fields in a form to populate, etc.).

[0129] In some instances, rather than outputting the command via the accessibility API, the assistant agent 608 outputs the command to perform the recommended action directly to one or more applications (e.g., executing on the user device 604) via one or more APIs respectively associated with the one or more applications.

[0130] Referring again to the example scenario described above, while the AI model 616 may have only received training data relating to a purchase order of a particular item and a particular quantity (and therefore having corresponding price lookup location in a database), the AI model 616 (as part of the training and through use of the accessibility information) may infer operations required to complete similar tasks having the same intended result. For example, when a user receives a second purchase order associated with a second item, a second quantity, and/or a second manufacturer, the AI model 616 may infer a second set of operations required to perform the task of completing the second purchase order that may differ from the exact set of operations that were performed based on the training data (e.g., the training data associated with the first purchase order) based on the different information associated with the

second purchase order (e.g., the second price, second manufacturer, second quantity, etc.). Based on the inferences of the AI model 616, the recommended action that defines a set of specific operations can therefore adapt to the context of the task at hand.

[0131] Additionally, in some instances, one or more operations defining a recommended action performed by the assistant agent 608 may be eliminated as compared to the set of operations defining the same action performed by a user. For example, the assistant agent 608 may perform calculations without rendering the steps of the calculation in a UI associated with calculator application of the user device 604. This aspect may be configurable by a user. For example, a user may configure one or more settings for the model 616 and the assistant agent 608 to control whether intermediate operations are replicated and output to user or whether only a final result of completing the task is provided. In some aspects, the AI model 616 and assistant agent 608 may be configured to output intermediate operations within an action and may prompt a user to confirm actions and/or provide feedback on actions and/or operations, which may be used as further training data for the AI model 616.

[0132] In some instances, the set of operations defining a recommended action may vary based on which applications, windows, or other UI elements, such as browser tabs, are rendered on the user device 604 prior to and during performance of the recommended action. For example, the set of operations defining a recommended action having a particular intended result may include outputting a command to open a portal or window associated with a particular database where the database is not already open on the user device 604. However, in instances where the database is already open, the set of operations defining the recommended action having the same intended result may not include outputting a command to open the database. Again, by having access to the accessibility information and similar information defining a current state of the user device 604, the AI model 616 and/or the agent 608 can identify operations within an initial recommended set of operations are not needed or need to be modified (e.g., to account for the current operating state of the user device 604).

[0133] FIG. 9 illustrates a second example workflow diagram 900 for training the AI model 616. As described above, the workflow includes obtaining accessibility data, wherein the accessibility data is generated by, for example, a platform executing on the user device 604, such as the OS 904 of the user device 604, during rendering a UI, such as the UI 908 illustrated in FIG. 9.

[0134] As described above, information identifying one or more UI elements in a UI of the user device 604, such as the UI 908, and/or information identifying one or more UI events in the UI 908 are received via calls made via the accessibility API (e.g., an accessibility API provided by accessibility services 912 of the user device 604). The accessibility services 912 may be provided by a platform rendering the UI 908, such as the OS of the user device 604 (e.g., through the OS kernel 916, OS core services 920, and OS UI frameworks 924).

[0135] The AI model 616 receives accessibility data 928 relating to UI elements and UI events in an application executing on the user device 604. The AI model 616 may receive some or all of the accessibility data 928 via an eventing manager 932 (e.g., an eventing manager 932 included as functionality of the assistant agent 608). For

example, the AI model **616** may receive the information relating to one or more UI events in the UI **908** via the eventing manager **932**. Alternatively or in addition, the AI model **616** may receive some or all of the accessibility data **928** via a layout manager **936** (e.g., a layout manager included as functionality of the assistant agent **608**). For example, the AI model **616** may receive information relating to UI elements in the UI **908** via the layout manager **936**.

[0136] The eventing and layout managers **932** and **936** may track and correlate events and UIs, respectively. For example, the layout manager **936** may track the layout of displayed UIs before and after events (e.g., how things are placed and where and in which context). Similarly, the eventing manager **932** may track sequence and context of events (e.g., UI events or other tracked events or data).

[0137] Information provided by the layout manager **936** and/or the eventing manager **932** may undergo preprocessing by the assistant agent **608** (e.g., at preprocessing block **938**) prior to being received by the AI model **616**. For example, the assistant agent **608** may process metadata associated with the one or more UI elements or UI events into a different format and correlate them to form patterns of tasks or actions which can be put together as tasks before providing the information as training data to the AI model **616**.

[0138] During operation (e.g., while a user interacts with the user device **604** and/or while applications execute on the user device **604**), the AI model **616** infers recommended actions to complete user tasks. As described above with reference to FIG. **8**, the AI model **616** may infer recommended actions responsive to a trigger condition such as, for example, a learned pattern behavior of a user (e.g., the user **940**). In some instances, the assistant agent **608**, with the AI model **616**, prompts the user **940** to confirm the recommended action, and performs the recommended action responsive to receiving user confirmation. In some instances, the AI model **616** infers a recommended action in response to receiving a prompt or query from the user **940** to complete a task.

[0139] In response to inferring a recommended action, the AI model **616** generates a token and provides the token to an accessibility and UI event generator **944** of, for example, the assistant agent **608**. Based on the token, the accessibility and UI event generator **944** generates commands to perform accessibility automation events and/or UI events. The tokens generated by the AI model **616** correspond to, for example, the operations that define recommended actions (e.g., keyboard clicks, window toggling, etc.). The accessibility and UI event generator **944** determines, based on the received tokens, whether commands are provided as UI events to the OS UI frameworks **924** of the user device **604** (e.g., as commands provided to applications executing on the user device **604**) or as automation events to the accessibility services (e.g., commands output as calls made via the accessibility API) of the user device **604**. In other words, operations on UI elements can be performed through the accessibility services **912** or through the operating system by, for example, writing (e.g., injecting text, clicking buttons, etc.) to the accessibility services or directly to the operating system. For example, accessibility services **912** may offer accessibility functionality that allows a user to enter text into a display UI using a microphone (as compared to via keyboard events) and the assistant action **608** may use this type of accessibility function to perform an action. In

situations where an application does not allow for automation events, user interactions can be mimicked directly through the displayed user interface. Also, some applications may provide an automation API that allows commands to be issued to the application even if the application is not open or active on the user device **604**.

[0140] The AI model **616** may infer the recommended action, the set of operations defining the recommended action, based on one or more stored recipes **948** associated with respective intended results of the action. In some instances, the assistant agent receives one or more stored recipes **948** for performing actions having a corresponding intended result as user input from the user **940**. The assistant agent may process the stored recipes **948** and corresponding intended results (e.g., at preprocessing block **938**) and may provide the stored recipes **948** as training data to the AI model **616**. Based on the stored recipes **948**, the AI model may infer recommended actions defined by different sets of operations than those included in the stored recipes **948** but associated with the same or similar intended results. The stored recipes **948** may allow user to customize or configure specific recommended actions for particular tasks, which, as noted above, may be provided to the AI model **616** as training data or otherwise processed by the AI model **616** to perform the operations defining those actions associated with the stored recipe for a particular task.

[0141] FIG. **10** illustrates an example method **1000** for training the AI model **616**. The method **1000** includes making (e.g., with the assistant agent **608**) a first call via an accessibility API to request accessibility data associated with a first UI displayed by the user device **604** (at block **1004**). The assistant agent **608** receives the first accessibility data via the accessibility API (at block **1008**), and identifies a first action performed by a user of the user device **604** (at block **1012**). The first action is associated with an intended result and defines a first set of operations, or steps.

[0142] The assistant agent **608** trains the AI model **616** based at least on the first accessibility data and the first set of operations (at block **1016**). The assistant agent **608** may also provide, as training data to the AI model **616**, contextual information such as time of day, day of the week, and/or the like such that the AI model **616** learns behavioral patterns of the user with respect to the user device **604**.

[0143] The method **1000** further includes making (e.g., with the assistant agent **608**) a second call via the accessibility API to request second accessibility data associated with a second UI displayed by the user device **604** (at block **1020**). The second UI displayed by the user device **604** may be a UI displayed at a different time than the first UI, or include different UI elements than the first UI. The assistant agent **608** determines, based on at least the second accessibility data and the AI model **616**, a recommended action defining a second set of operations (at block **1024**). For example, based on the accessibility data and the AI model **616**, the assistant agent may identify a trigger condition for performing a recommended action (e.g., determining the user received a particular type of email, determining that the time of day or day of week indicates that a certain task must be completed, determining that the user has launched a particular application, and/or the like). The recommended action defines a second set of operations. As described above, in some instances, the first set of operations differs from the second set of operations, even when an intended result of the recommended action is the same as the first

action. However, in some instances, the first set of operations is the same as the second set of operations.

[0144] The assistant agent 608 outputs a command to perform the recommended action on the user device 604 (at block 1028). As described above, the assistant agent 608 may output the command to perform the recommended action by making a third call via the accessibility API to the user device 604 (e.g., to cause results and/or inputs associated with the recommended action or second set of operations to render on the second UI of the user device 604). In some instances, the assistant agent 608 outputs the command to at least one application executing on the user device 604, for example, by making a call to an API associated with the at least one application executing on the user device 604.

[0145] In some instances, the assistant agent 608 generates a notification (e.g., a visual or audio notification) to the user suggesting the recommended action, and performs the recommended action (e.g., at block 1028) responsive to receiving a user confirmation of the recommended action.

[0146] Although specific embodiments have been described, various modifications, alterations, alternative constructions, and equivalents are also encompassed within the scope of the disclosure. Embodiments are not restricted to operation within certain specific data processing environments but are free to operate within a plurality of data processing environments. Additionally, although embodiments have been described using a particular series of transactions and steps, it should be apparent to those skilled in the art that the scope of the present disclosure is not limited to the described series of transactions and steps. Various features and aspects of the above-described embodiments may be used individually or jointly.

[0147] Further, while embodiments have been described using a particular combination of hardware and software, it should be recognized that other combinations of hardware and software are also within the scope of the present disclosure. Embodiments may be implemented only in hardware, or only in software, or using combinations thereof. The various processes described herein can be implemented on the same processor or different processors in any combination. Accordingly, where components or services are described as being configured to perform certain operations, such configuration can be accomplished, e.g., by designing electronic circuits to perform the operation, by programming programmable electronic circuits (such as microprocessors) to perform the operation, or any combination thereof. Processes can communicate using a variety of techniques including but not limited to conventional techniques for inter process communication, and different pairs of processes may use different techniques, or the same pair of processes may use different techniques at different times.

[0148] The specification and drawings are, accordingly, to be regarded in an illustrative rather than a restrictive sense. It will, however, be evident that additions, subtractions, deletions, and other modifications and changes may be made thereunto without departing from the broader spirit and scope as set forth in the claims. Thus, although specific disclosure embodiments have been described, these are not intended to be limiting. Various modifications and equivalents are within the scope of the following claims.

[0149] The use of the terms “a” and “an” and “the” and similar referents in the context of describing the disclosed embodiments (especially in the context of the following claims) are to be construed to cover both the singular and the

plural, unless otherwise indicated herein or clearly contradicted by context. The terms “comprising,” “having,” “including,” and “containing” are to be construed as open-ended terms (i.e., meaning “including, but not limited to,”) unless otherwise noted. The term “connected” is to be construed as partly or wholly contained within, attached to, or joined together, even if there is something intervening. Recitation of ranges of values herein are merely intended to serve as a shorthand method of referring individually to each separate value falling within the range, unless otherwise indicated herein and each separate value is incorporated into the specification as if it were individually recited herein. All methods described herein can be performed in any suitable order unless otherwise indicated herein or otherwise clearly contradicted by context. The use of any and all examples, or exemplary language (e.g., “such as”) provided herein, is intended merely to better illuminate embodiments and does not pose a limitation on the scope of the disclosure unless otherwise claimed. No language in the specification should be construed as indicating any non-claimed element as essential to the practice of the disclosure.

[0150] Disjunctive language such as the phrase “at least one of X, Y, or Z,” unless specifically stated otherwise, is intended to be understood within the context as used in general to present that an item, term, etc., may be either X, Y, or Z, or any combination thereof (e.g., X, Y, and/or Z). Thus, such disjunctive language is not generally intended to, and should not, imply that certain embodiments require at least one of X, at least one of Y, or at least one of Z to each be present.

[0151] Preferred embodiments of this disclosure are described herein, including the best mode known for carrying out the disclosure. Variations of those preferred embodiments may become apparent to those of ordinary skill in the art upon reading the foregoing description. Those of ordinary skill should be able to employ such variations as appropriate and the disclosure may be practiced otherwise than as specifically described herein. Accordingly, this disclosure includes all modifications and equivalents of the subject matter recited in the claims appended hereto as permitted by applicable law. Moreover, any combination of the above-described elements in all possible variations thereof is encompassed by the disclosure unless otherwise indicated herein.

[0152] All references, including publications, patent applications, and patents, cited herein are hereby incorporated by reference to the same extent as if each reference were individually and specifically indicated to be incorporated by reference and were set forth in its entirety herein.

[0153] In the foregoing specification, aspects of the disclosure are described with reference to specific embodiments thereof, but those skilled in the art will recognize that the disclosure is not limited thereto. Various features and aspects of the above-described disclosure may be used individually or jointly. Further, embodiments can be utilized in any number of environments and applications beyond those described herein without departing from the broader spirit and scope of the specification. The specification and drawings are, accordingly, to be regarded as illustrative rather than restrictive.

What is claimed is:

1. An electronic device comprising:
 - at least one electronic processor configured to:
 - make a first call via an accessibility application programming interface (“API”) to request first accessibility data associated with a first user interface (“UI”) displayed by a user device;
 - receive the first accessibility data via the accessibility API, the first accessibility data comprising at least one of (a) information identifying one or more UI elements in the first UI or (b) information identifying one or more UI events in the first UI;
 - identify, based on at least the first accessibility data, a first action performed by a user of the user device, the first action defining a first set of operations;
 - train, based on at least the first accessibility data and the first set of operations, an artificial intelligence (“AI”) model;
 - make a second call via the accessibility API to request second accessibility data associated with a second UI displayed by the user device;
 - determine, based on at least the second accessibility data and the AI model, a recommended action, the recommend action defining a second set of operations; and
 - output a command to perform the recommended action on the user device.
2. The electronic device of claim 1, wherein the first set of operations differs from the second set of operations.
3. The electronic device of claim 1, wherein the at least one electronic processor is configured to output the command to perform the recommended action by making a third call via the accessibility API to the user device.
4. The electronic device of claim 1, wherein the at least one electronic processor is configured to output the command to at least one application executed on the user device.
5. The electronic device of claim 4, wherein the at least one electronic processor is configured to output the command by making a call to an API associated with the at least one application executed on the user device.
6. The electronic device of claim 1, wherein the information identifying the one or more UI elements in the first UI includes at least one selected from the group consisting of: (a) a respective element type of the one or more UI elements in the first UI, (b) a respective identifier of the one or more UI elements in the first UI, or (c) a respective state or condition of the one or more UI elements in the first UI.
7. The electronic device of claim 6, wherein the state or condition of the one or more UI elements in the first UI includes at least one selected from the group consisting of pixel coordinates of the one or more UI elements, dimensions of the one or more UI elements, and an active or passive state of the one or more UI elements.
8. The electronic device of claim 1, wherein the information identifying the one or more UI elements in the first UI is organized as a hierarchical tree.
9. The electronic device of claim 8, wherein
 - the one or more UI elements in the first UI comprises a first UI element that includes a second UI element;
 - the hierarchical tree comprises a first hierarchical level that is above a second hierarchical level;

the first hierarchical level includes a first node representing the first UI element; and

the second hierarchical level includes a second node representing the second UI element.

10. The electronic device of claim 1, wherein the information identifying the one or more UI events in the first UI includes one or more notifications of changes in states or conditions of the UI elements in the first UI.

11. The electronic device of claim 10, wherein the changes in states or conditions of the UI elements in the first UI are caused by user interaction with the UI elements.

12. The electronic device of claim 1, wherein the first call made via the accessibility API is made to a platform rendering the first UI.

13. The electronic device of claim 12, wherein the platform includes one of an operating system executing on the first device or a browser application executing on the first device.

14. The electronic device of claim 1, wherein the first accessibility data is generated based on metadata associated with the one or more UI elements in the first UI, the metadata being exposed via the accessibility API.

15. The electronic device of claim 14, wherein the metadata is specified in one of (a) application code of application for which the first UI is being rendered or (b) content code of web content for which the first UI is being rendered.

16. The electronic device of claim 14, wherein the metadata specifies hierarchical relationships between the one or more UI elements in the first UI.

17. The electronic device of claim 14, wherein the metadata of the first UI elements includes at least one selected from a group consisting of a type of each respective first UI element, an active or passive status of each respective first UI element, pixel coordinates of each respective first UI element, and a displayed label of each respective first UI element.

18. The electronic device of claim 1, wherein the first accessibility data is generated by a platform executing on the first device during rendering of the first UI.

19. A method for training an artificial intelligence model, the method comprising:

- making a first call via an accessibility application programming interface (“API”) to request first accessibility data associated with a first user interface (“UI”) displayed by a user device;

- receiving the first accessibility data via the accessibility API, the first accessibility data comprising at least one of (a) information identifying one or more UI elements in the first UI or (b) information identifying one or more UI events in the first UI;

- identifying, based on at least the first accessibility data, a first action performed by a user of the user device, the first action defining a first set of operations;

- training, based on at least the first accessibility data and the first set of operations, an artificial intelligence (“AI”) model;

- making a second call via the accessibility API to request second accessibility data associated with a second UI displayed by the user device;

- determining, based on at least the second accessibility data and the AI model, a recommended action, the recommend action defining a second set of operations; and

outputting a command to perform the recommended action on the user device.

20. The method of claim **19**, wherein the first set of operations differs from the second set of operations.

* * * * *